Hazardous Celestial Objects: Full Analysis

Alejandro Arroyo, Ryujin Bupp, Jose Placeres, Fernando Ricaurte

STA 5703 Data Mining Methodology, University of Central Florida, Spring 2026

Contents

1 Libraries **1**

2 Data Cleaning and Preprocessing **1**
2.1 Loading the Raw Dataset . . . 2
2.2 Removing Missing Values . . . 2
2.3 Removing Redundant Columns . . . 2
2.4 Filtering to Earth-Orbiting Objects . . . 2
2.5 Removing Duplicate Objects . . . 2
2.6 Final Null Check . . . 2
2.7 Loading the Cleaned Dataset (if the raw file is absent) . . . 3

3 Exploratory Data Analysis **3**

4 Variable Selection: Logistic Regression, Ridge, and LASSO **9**
4.1 Data Preparation . . . 9
4.2 Train / Validation / Test Split . . . 10
4.3 Baseline Logistic Regression . . . 11
4.4 Best Subset Selection . . . 11
4.5 Stepwise Selection (Bidirectional) . . . 13
4.6 Ridge Regression . . . 14
4.7 LASSO Regression . . . 17
4.8 Step 2 Model Comparison (Validation Set) . . . 19

5 Ensemble Methods: Random Forest, Boosting, and Bagging **21**
5.1 Random Forest . . . 21
5.2 Gradient Boosting . . . 27
5.3 Bagging . . . 31

6 Model Comparison **38**
6.1 Confusion Matrices . . . 38
6.2 ROC Curve Comparison . . . 39
6.3 Metrics Comparison Table . . . 39
6.4 Variable Importance Comparison . . . 40
6.5 All Models on the Test Set . . . 41

7 Session Info **43**

About this version. This is the team's final analysis with two refinements: (1) every model, from logistic regression to boosting, is evaluated on the same validation and test rows (Section 4.2), and (2) the Random Forest threshold is read from the validation ROC curve rather than typed in, so the results reproduce on any machine (Section 5.1).

1 Libraries

```
library(dplyr)
library(ggplot2)
library(tidyr)
library(knitr)
library(kableExtra)
library(readxl)
library(caret)
library(randomForest)
library(pROC)
library(glmnet)
library(leaps)
library(MASS)
library(gbm)
library(scales)
library(tibble)
library(corrplot)
library(GGally)

# MASS also defines select(); make sure dplyr's version is used
select <- dplyr::select
```

2 Data Cleaning and Preprocessing

The raw data is the NASA NeoWs 1950–2025 dataset (375,659 rows, 41 columns), available on Kaggle as Near Earth Comets Classification.

If the raw file `data/nasa_neows_1950_2025.csv` is present, the cleaning steps below run and produce the cleaned dataset. If it is not, the notebook loads the already-cleaned file `data/cleaned_neows_1950_2025.xlsx` (35,430 rows), which is the output of these same steps.

```
raw_path   <- "data/nasa_neows_1950_2025.csv"
clean_path <- "data/cleaned_neows_1950_2025.xlsx"
have_raw   <- file.exists(raw_path)
have_raw
```

```
## [1] FALSE
```

2.1 Loading the Raw Dataset

```
asteroids <- read.csv(raw_path, header = TRUE, stringsAsFactors = FALSE)
str(asteroids)
```

2.2 Removing Missing Values

```
asteroids_clean <- na.omit(asteroids)
nrow(asteroids_clean)
```

2.3 Removing Redundant Columns

Name and Equinox are dropped in favor of Neo Reference ID, and duplicate unit conversions are dropped to keep a single unit of measurement per quantity.

```r
asteroids_clean <- asteroids_clean[, !(names(asteroids_clean) %in% c("Name", "Equinox"))]

superfluous <- c(
  "Est.Dia.in.KM.min.", "Est.Dia.in.M.min.", "Est.Dia.in.M.max.",
  "Est.Dia.in.Miles.min.", "Est.Dia.in.Miles.max.",
  "Est.Dia.in.Feet.min.", "Est.Dia.in.Feet.max.",
  "Relative.Velocity.km.per.sec", "Miles.per.hour",
  "Miss.Dist..Astronomical.", "Miss.Dist..lunar.", "Miss.Dist..miles."
)
asteroids_clean <- asteroids_clean[, !(names(asteroids_clean) %in% superfluous)]
colnames(asteroids_clean)
```

2.4 Filtering to Earth-Orbiting Objects

```r
unique(asteroids_clean$Orbiting.Body)
asteroids_clean <- asteroids_clean[asteroids_clean$Orbiting.Body == "Earth", ]
unique(asteroids_clean$Orbiting.Body)
```

2.5 Removing Duplicate Objects

Each asteroid appears once per close approach. Keeping one row per Neo Reference ID prevents the same object from landing in both the training and test sets (data leakage).

```r
sum(duplicated(asteroids_clean$Neo.Reference.ID))
asteroids_clean <- asteroids_clean[!duplicated(asteroids_clean$Neo.Reference.ID), ]
sum(duplicated(asteroids_clean$Neo.Reference.ID))
```

2.6 Final Null Check

```r
asteroids_clean <- na.omit(asteroids_clean)
colSums(is.na(asteroids_clean))
```

2.7 Loading the Cleaned Dataset (if the raw file is absent)

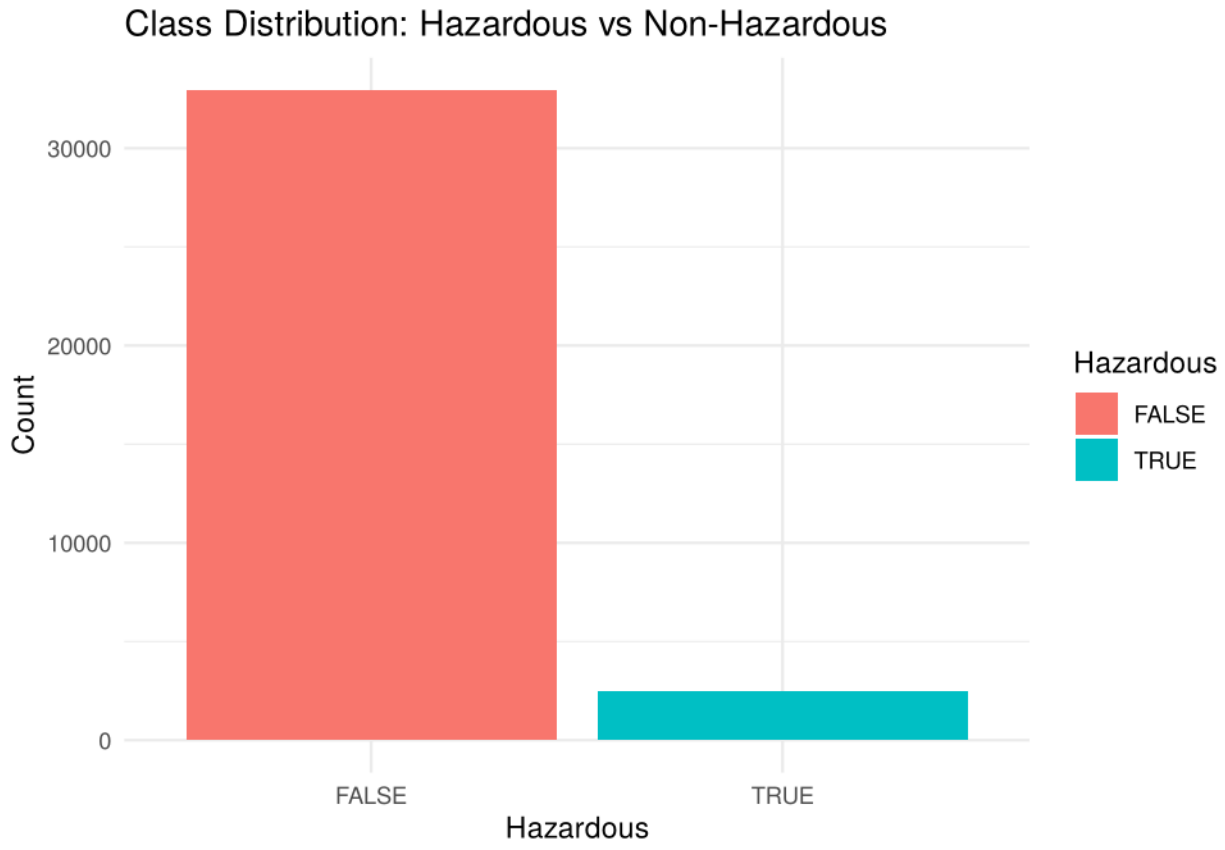
```r
asteroids_clean <- as.data.frame(read_excel(clean_path))
# One value is stored as text in scientific notation; convert it to numeric
asteroids_clean$Minimum.Orbit.Intersection <-
  as.numeric(asteroids_clean$Minimum.Orbit.Intersection)
asteroids_clean$Miss.Dist..kilometers. <-
  as.numeric(asteroids_clean$Miss.Dist..kilometers.)
str(asteroids_clean)
```

```
## 'data.frame':    35430 obs. of  26 variables:
##  $ Date                       : POSIXct, format: "1900-01-01" "1900-01-01" ...
##  $ Neo.Reference.ID           : num  3102683 3285299 3653973 3703077 3771605 ...
##  $ Absolute.Magnitude         : num  22.1 20.3 20.4 24.7 21.8 ...
##  $ Est.Dia.in.KM.max.         : num  0.226 0.5177 0.5012 0.0682 0.2594 ...
##  $ Close.Approach.Date        : POSIXct, format: "1900-01-02" "1900-01-01" ...
##  $ Epoch.Date.Close.Approach  : num  -2.21e+12 -2.21e+12 -2.21e+12 -2.21e+12 -2.21e+12 ...
##  $ Relative.Velocity.km.per.hr: num  47370 64506 49354 46707 35489 ...
##  $ Miss.Dist..kilometers.     : num  70707713 62079141 25363974 39688629 57934724 ...
##  $ Orbit.ID                   : num  13 53 17 16 11 11 53 79 13 2 ...
```

```
##  $ Orbit.Determination.Date   : POSIXct, format: "2021-04-14 21:15:00" "2021-04-14 22:45:00" ...
##  $ Orbit.Uncertainty          : num  1 0 1 3 3 1 0 0 0 8 ...
##  $ Minimum.Orbit.Intersection : num  0.0113 0.026 0.085 0.0349 0.0549 ...
##  $ Jupiter.Tisserand.Invariant: num  5.9 4.22 4.49 5.48 6.67 ...
##  $ Epoch.Osculation           : num  2460200 2460200 2460200 2460200 2460200 ...
##  $ Eccentricity               : num  0.557 0.429 0.589 0.16 0.381 ...
##  $ Semi.Major.Axis            : num  1.006 1.567 1.425 1.137 0.876 ...
##  $ Inclination                : num  0.849 25.37 5.815 11.277 16.125 ...
##  $ Asc.Node.Longitude         : num  127.1 321 36.2 285.4 149.2 ...
##  $ Orbital.Period             : num  369 716 621 443 299 ...
##  $ Perihelion.Distance        : num  0.446 0.895 0.585 0.955 0.542 ...
##  $ Perihelion.Arg             : num  186.8 58.4 153.8 240.1 244.2 ...
##  $ Aphelion.Dist              : num  1.57 2.24 2.26 1.32 1.21 ...
##  $ Perihelion.Time            : num  2460307 2460091 2460421 2460183 2460128 ...
##  $ Mean.Anomaly               : num  255.7 55 232 13.9 87.8 ...
##  $ Mean.Motion                : num  0.976 0.503 0.58 0.813 1.203 ...
##  $ Hazardous                  : logi  TRUE TRUE FALSE FALSE FALSE TRUE ...
```

3 Exploratory Data Analysis

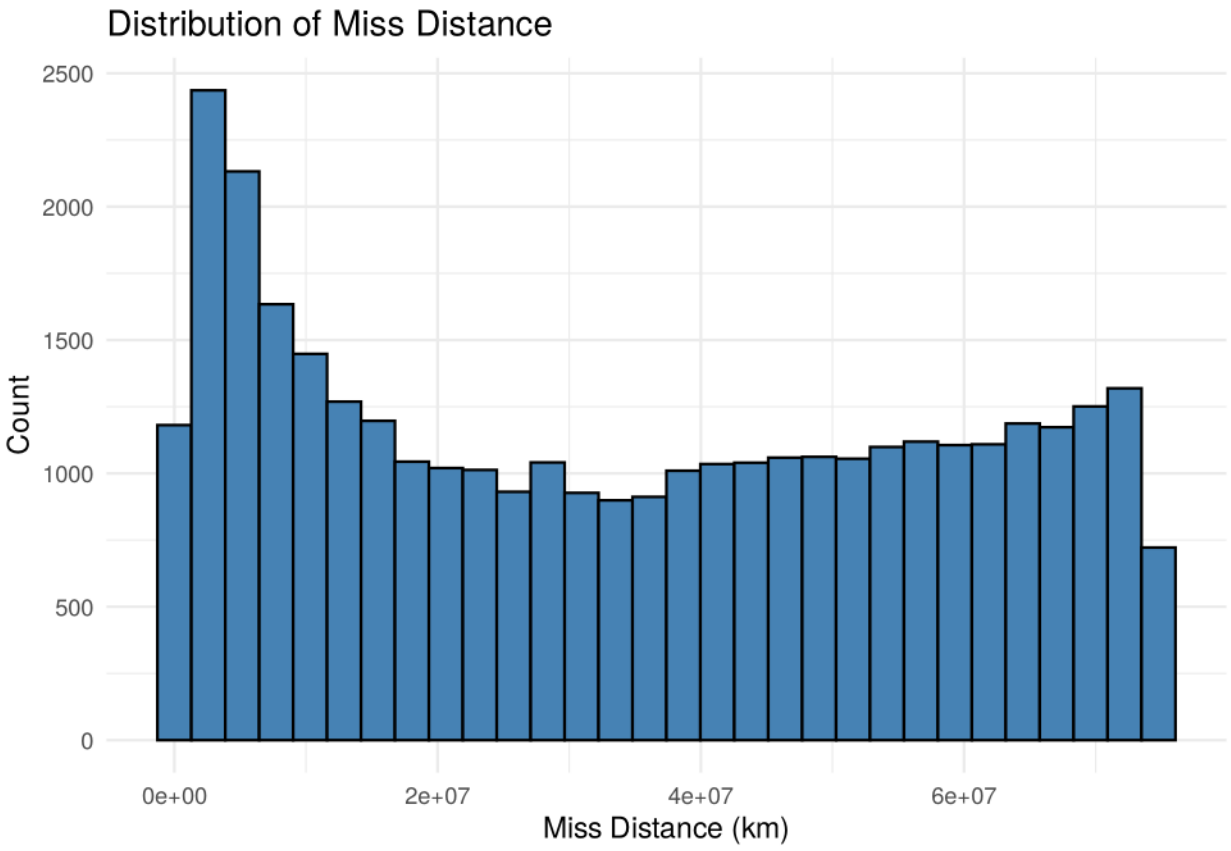
```
ggplot(asteroids_clean, aes(x = as.factor(Hazardous), fill = as.factor(Hazardous))) +
  geom_bar() +
  theme_minimal() +
  labs(title = "Class Distribution: Hazardous vs Non-Hazardous",
       x = "Hazardous", y = "Count", fill = "Hazardous")
```



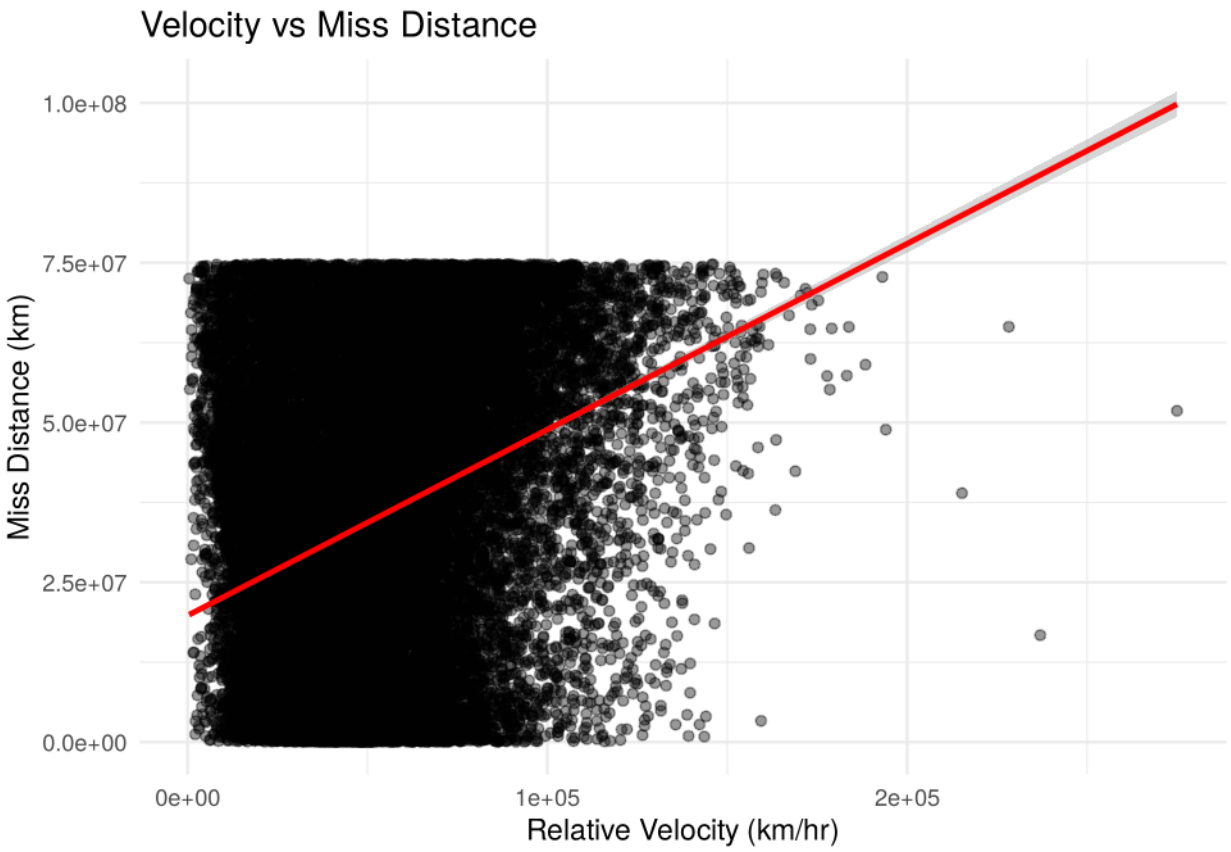
```r
prop.table(table(asteroids_clean$Hazardous))
```

```
##
##      FALSE       TRUE
## 0.92955123 0.07044877
```

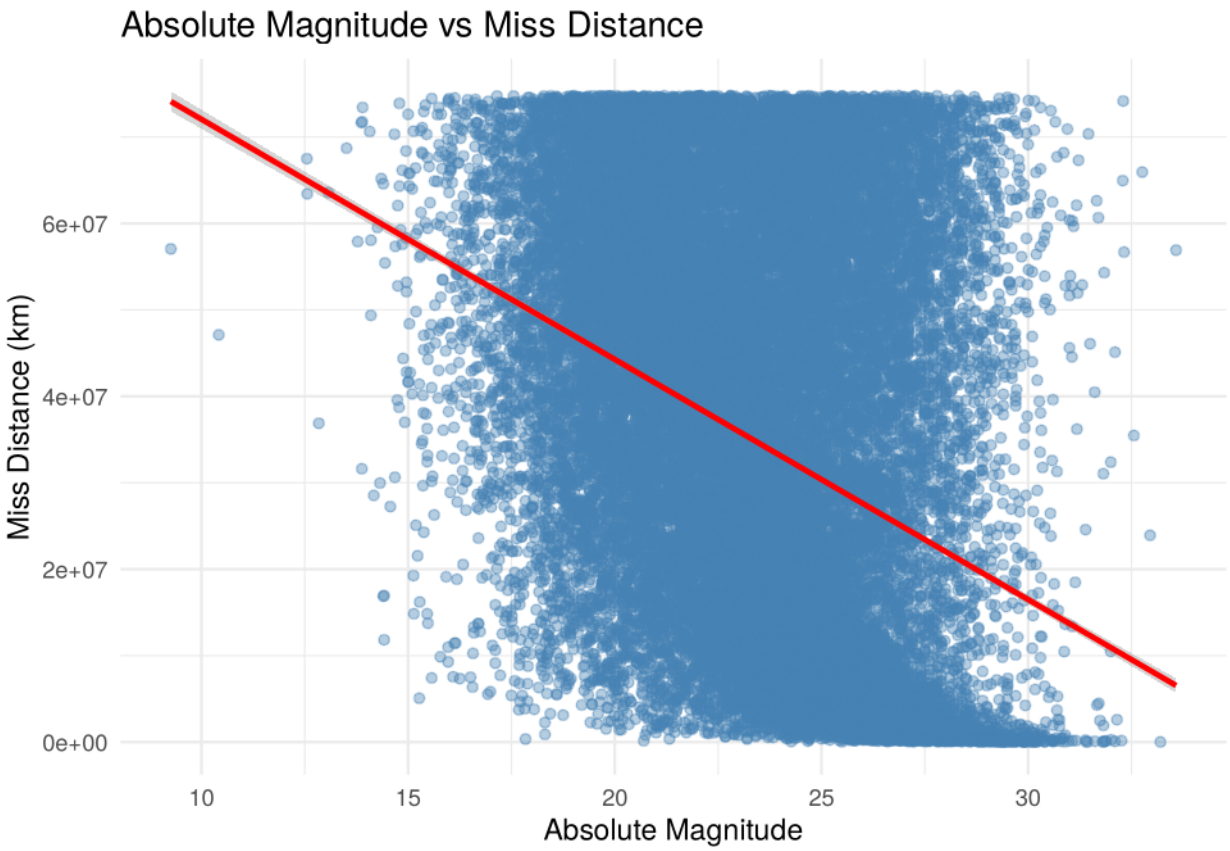
```r
ggplot(asteroids_clean, aes(x = Miss.Dist..kilometers.)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "black") +
  theme_minimal() +
  labs(title = "Distribution of Miss Distance", x = "Miss Distance (km)", y = "Count")
```




```r
ggplot(asteroids_clean, aes(x = Relative.Velocity.km.per.hr, y = Miss.Dist..kilometers.)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", color = "red") +
  theme_minimal() +
  labs(title = "Velocity vs Miss Distance",
       x = "Relative Velocity (km/hr)", y = "Miss Distance (km)")
```

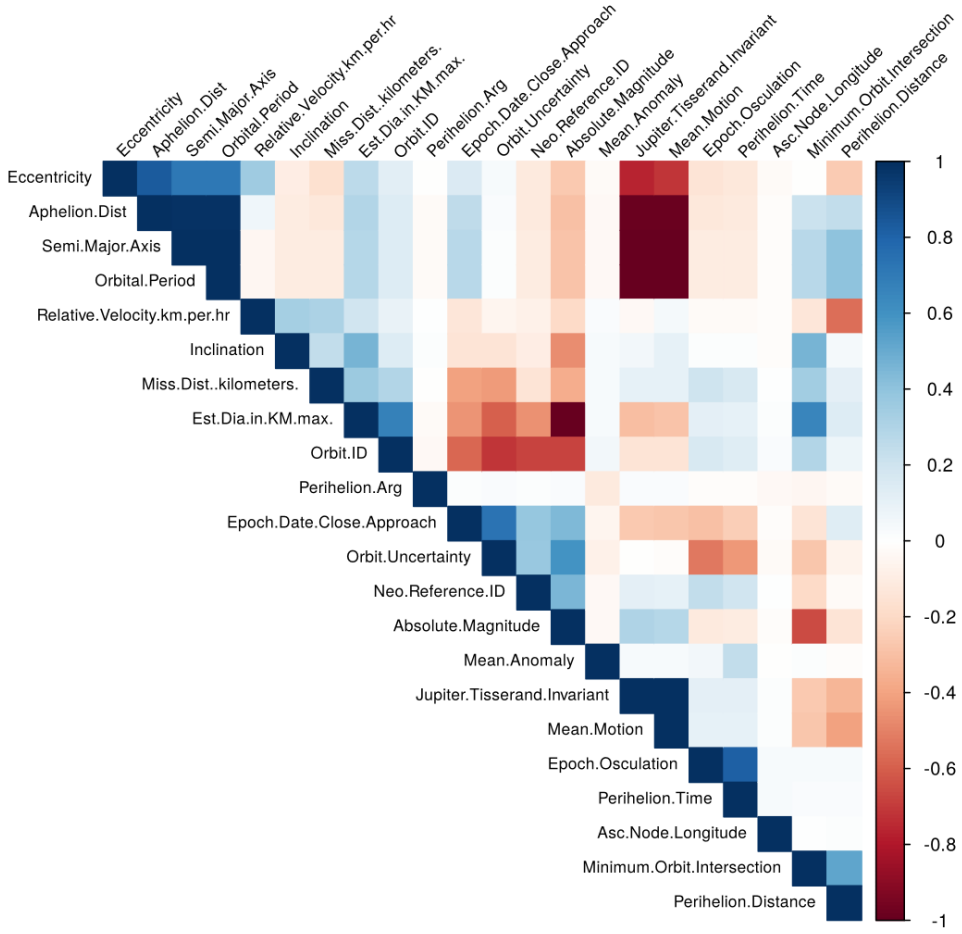



```
ggplot(asteroids_clean, aes(x = Absolute.Magnitude, y = Miss.Dist..kilometers.)) +
  geom_point(alpha = 0.4, color = "steelblue") +
  geom_smooth(method = "lm", color = "red") +
  theme_minimal() +
  labs(title = "Absolute Magnitude vs Miss Distance",
       x = "Absolute Magnitude", y = "Miss Distance (km)")
```

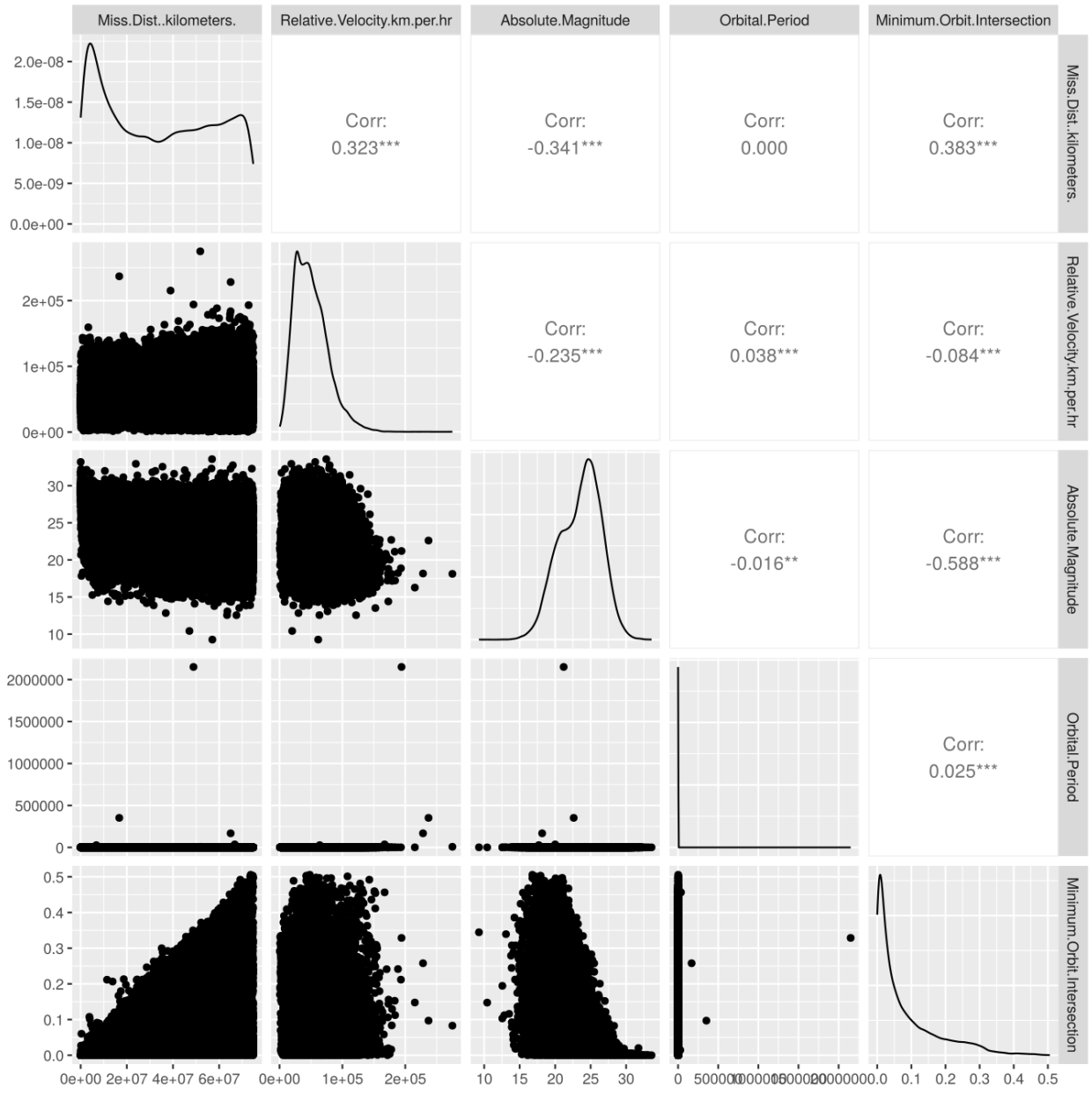


Spearman correlation is used because several variables are heavily skewed.

```
numeric_data <- select_if(asteroids_clean, is.numeric)
cor_matrix <- cor(numeric_data, method = "spearman", use = "complete.obs")
corrplot(cor_matrix, method = "color", type = "upper", order = "hclust",
         tl.col = "black", tl.cex = 0.7, tl.srt = 45)
```




```
subset_vars <- asteroids_clean %>%
  dplyr::select(Miss.Dist..kilometers., Relative.Velocity.km.per.hr,
                Absolute.Magnitude, Orbital.Period, Minimum.Orbit.Intersection)
ggpairs(subset_vars)
```



4 Variable Selection: Logistic Regression, Ridge, and LASSO

4.1 Data Preparation

```
# Drop all non-predictive columns once for use across all models
shared_drop <- c("Date", "Neo.Reference.ID", "Close.Approach.Date",
                 "Orbit.Determination.Date", "Epoch.Date.Close.Approach",
                 "Orbit.ID", "Perihelion.Time", "Orbiting.Body")
model_data <- asteroids_clean[, !(names(asteroids_clean) %in% shared_drop)]

# Convert all predictors to numeric
predictor_names <- setdiff(names(model_data), "Hazardous")
model_data <- model_data %>%
```

```
  mutate(across(all_of(predictor_names), ~ as.numeric(as.character(.))))

# hco_data: numeric Hazardous (1/0) for logistic regression, Ridge, LASSO
hco_data <- model_data
hco_data$Hazardous <- ifelse(as.character(model_data$Hazardous) %in% c("True", "TRUE"), 1, 0)

# comets_model: factor Hazardous for Random Forest, Boosting, Bagging
comets_model <- model_data
comets_model$Hazardous <- factor(
  ifelse(as.character(model_data$Hazardous) %in% c("True", "TRUE"), "TRUE", "FALSE"),
  levels = c("FALSE", "TRUE")
)
```

4.2 Train / Validation / Test Split

A single stratified 60/20/20 split is shared by every model.

> **Correction (post-course).** In the original notebook, the Step 2 sets were built with `hco_data[val_idx, ]` and `hco_data[-c(train_idx, val_idx), ]`. But `val_idx` holds row positions *within* `temp` (the 40% held out from training), not within the full dataset, so the Step 2 validation and test sets overlapped the training set. The lines below take `val_idx` from the held-out rows, so Step 2 now uses exactly the same validation and test rows as the ensemble models.

```
set.seed(5703)
train_idx   <- createDataPartition(comets_model$Hazardous, p = 0.6, list = FALSE)
comet_train <- comets_model[train_idx, ]
temp        <- comets_model[-train_idx, ]

val_idx    <- createDataPartition(temp$Hazardous, p = 0.5, list = FALSE)
comet_val  <- temp[val_idx, ]
comet_test <- temp[-val_idx, ]

minority_n <- min(table(comet_train$Hazardous))

# Same rows for the Step 2 models (corrected indexing)
hco_heldout         <- hco_data[-train_idx, ]
train_hco_data      <- hco_data[train_idx, ]
validation_hco_data <- hco_heldout[val_idx, ]
test_hco_data       <- hco_heldout[-val_idx, ]

# Sanity checks: no overlap, and the two versions line up row for row
stopifnot(nrow(train_hco_data) + nrow(validation_hco_data) + nrow(test_hco_data) == nrow(hco_data))
stopifnot(all(validation_hco_data$Hazardous == as.numeric(comet_val$Hazardous == "TRUE")))
stopifnot(all(test_hco_data$Hazardous == as.numeric(comet_test$Hazardous == "TRUE")))

c(train = nrow(comet_train), validation = nrow(comet_val), test = nrow(comet_test))
```

```
##      train validation       test
##      21259       7086       7085
```

```
table(comet_train$Hazardous)
```

```
##
## FALSE  TRUE
```

```
## 19761  1498
```

4.3 Baseline Logistic Regression

```
baseline_model_hco_data <- glm(Hazardous ~ ., data = train_hco_data, family = binomial)
summary(baseline_model_hco_data)
```

```
##
## Call:
## glm(formula = Hazardous ~ ., family = binomial, data = train_hco_data)
##
## Coefficients:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  8.390e+00  1.137e+02   0.074  0.94120
## Absolute.Magnitude          -2.957e+00  1.079e-01 -27.396  < 2e-16 ***
## Est.Dia.in.KM.max.          -5.073e+00  2.556e-01 -19.846  < 2e-16 ***
## Relative.Velocity.km.per.hr -8.712e-07  3.453e-06  -0.252  0.80079
## Miss.Dist..kilometers.       3.801e-09  3.073e-09   1.237  0.21614
## Orbit.Uncertainty           -7.453e-02  2.526e-02  -2.951  0.00317 **
## Minimum.Orbit.Intersection  -1.178e+02  3.988e+00 -29.529  < 2e-16 ***
## Jupiter.Tisserand.Invariant  3.462e-01  1.419e+00   0.244  0.80729
## Epoch.Osculation             2.396e-05  4.635e-05   0.517  0.60519
## Eccentricity                 3.824e-01  1.248e+00   0.306  0.75928
## Semi.Major.Axis             -3.066e+07  1.552e+08  -0.198  0.84334
## Inclination                  9.430e-03  1.024e-02   0.921  0.35704
## Asc.Node.Longitude          -1.227e-03  5.322e-04  -2.306  0.02110 *
## Orbital.Period              -2.421e-04  8.516e-04  -0.284  0.77617
## Perihelion.Distance          1.533e+07  7.758e+07   0.198  0.84334
## Perihelion.Arg              -1.680e-04  5.483e-04  -0.306  0.75923
## Aphelion.Dist                1.533e+07  7.758e+07   0.198  0.84334
## Mean.Anomaly                 5.348e-04  4.960e-04   1.078  0.28098
## Mean.Motion                 -1.252e+00  4.498e+00  -0.278  0.78081
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 10835.2  on 21258  degrees of freedom
## Residual deviance:  2202.3  on 21240  degrees of freedom
## AIC: 2240.3
##
## Number of Fisher Scoring iterations: 15
```

`Aphelion.Dist` has an `NA` coefficient because it is an exact linear combination of the semi-major axis and perihelion distance.

4.4 Best Subset Selection

```
best_subset_hco_data <- regsubsets(Hazardous ~ ., data = train_hco_data,
                                   nvmax = 19, method = "exhaustive")
```

```
## Reordering variables and trying again:
```

```
summary_best_subset_hco_data <- summary(best_subset_hco_data)
best_model_bic <- which.min(summary_best_subset_hco_data$bic)
```

```r
coefficients_best_model <- coef(best_subset_hco_data, best_model_bic)

kable(
  data.frame(
    Vars  = 1:length(summary_best_subset_hco_data$adjr2),
    AdjR2 = round(summary_best_subset_hco_data$adjr2, 4),
    BIC   = round(summary_best_subset_hco_data$bic, 4),
    CP    = round(summary_best_subset_hco_data$cp, 4)
  ),
  caption = "Best Subset Selection Performance Metrics",
  col.names = c("Number of Variables", "Adj R-Squared", "BIC", "Mallows' Cp"),
  align = "c"
) %>% kable_styling(position = "left", full_width = FALSE)
```

Table 1: Best Subset Selection Performance Metrics

Number of Variables	Adj R-Squared	BIC	Mallows' Cp
1	0.1034	-2302.444	6509.1003
2	0.2910	-7282.492	702.5322
3	0.3052	-7703.613	263.8057
4	0.3110	-7872.622	85.4394
5	0.3124	-7908.198	41.7858
6	0.3129	-7915.166	26.8367
7	0.3132	-7913.588	20.4465
8	0.3133	-7909.845	16.2244
9	0.3135	-7905.991	12.1158
10	0.3137	-7903.048	7.0974
11	0.3137	-7894.811	7.3713
12	0.3137	-7886.139	8.0799
13	0.3137	-7876.967	9.2878
14	0.3137	-7867.105	11.1853
15	0.3137	-7857.238	13.0879
16	0.3136	-7847.356	15.0054
17	0.3136	-7837.397	17.0000

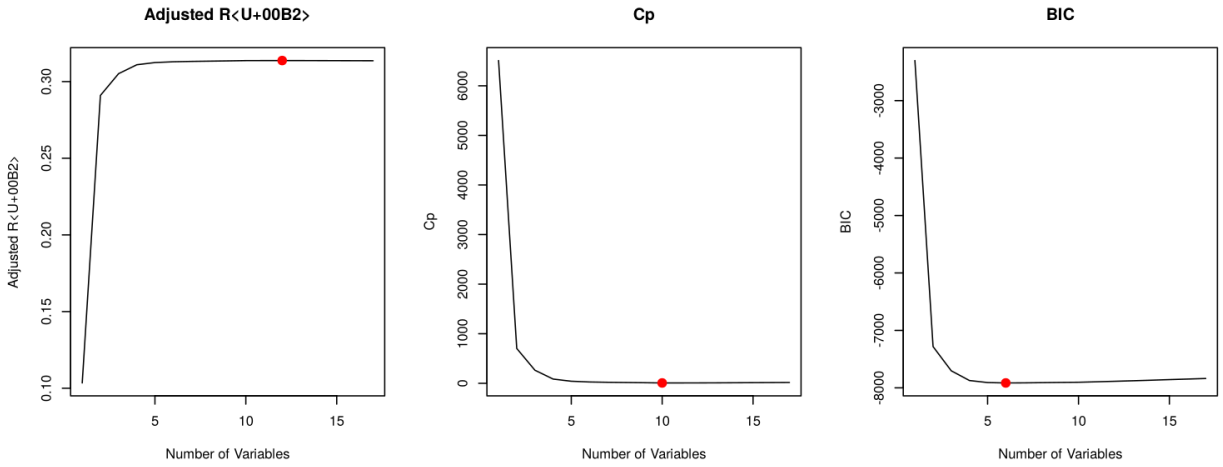
```r
kable(
  data.frame(Variable = sub("(Intercept)", "Intercept", names(coefficients_best_model), fixed = TRUE),
             Coefficient = round(as.numeric(coefficients_best_model), 6)),
  caption = paste("Variables Selected by Best Subset Selection (Size =", best_model_bic, ")"),
  col.names = c("Variable Name", "Estimated Coefficient"),
  align = "l"
) %>% kable_styling(position = "left", full_width = FALSE)
```

Table 2: Variables Selected by Best Subset Selection (Size = 6)

Variable Name	Estimated Coefficient
Intercept	1.457112
Absolute.Magnitude	-0.048075
Est.Dia.in.KM.max.	-0.038009
Orbit.Uncertainty	-0.015320

Minimum.Orbit.Intersection	-1.519366
Jupiter.Tisserand.Invariant	-0.011390
Inclination	0.000669

```
par(mfrow = c(1, 3))
plot(summary_best_subset_hco_data$adjr2, xlab = "Number of Variables",
     ylab = "Adjusted R²", type = "l", main = "Adjusted R²")
points(which.max(summary_best_subset_hco_data$adjr2),
       max(summary_best_subset_hco_data$adjr2), col = "red", cex = 2, pch = 20)
plot(summary_best_subset_hco_data$cp, xlab = "Number of Variables",
     ylab = "Cp", type = "l", main = "Cp")
points(which.min(summary_best_subset_hco_data$cp),
       min(summary_best_subset_hco_data$cp), col = "red", cex = 2, pch = 20)
plot(summary_best_subset_hco_data$bic, xlab = "Number of Variables",
     ylab = "BIC", type = "l", main = "BIC")
points(which.min(summary_best_subset_hco_data$bic),
       min(summary_best_subset_hco_data$bic), col = "red", cex = 2, pch = 20)
```




```
par(mfrow = c(1, 1))
```

4.5 Stepwise Selection (Bidirectional)

```
train_hco_data$Hazardous      <- as.factor(train_hco_data$Hazardous)
validation_hco_data$Hazardous <- as.factor(validation_hco_data$Hazardous)
test_hco_data$Hazardous       <- as.factor(test_hco_data$Hazardous)

full_model_hco_data <- glm(Hazardous ~ ., data = train_hco_data, family = binomial)
stepwise_hco_data   <- stepAIC(full_model_hco_data, direction = "both", trace = FALSE)
summary(stepwise_hco_data)
```

```
##
## Call:
## glm(formula = Hazardous ~ Absolute.Magnitude + Est.Dia.in.KM.max. +
##     Orbit.Uncertainty + Minimum.Orbit.Intersection + Asc.Node.Longitude,
##     family = binomial, data = train_hco_data)
##
```

```
## Coefficients:
##                               Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  6.957e+01  2.398e+00  29.009  < 2e-16 ***
## Absolute.Magnitude          -2.994e+00  1.045e-01 -28.659  < 2e-16 ***
## Est.Dia.in.KM.max.          -5.107e+00  2.536e-01 -20.134  < 2e-16 ***
## Orbit.Uncertainty           -6.938e-02  2.043e-02  -3.397 0.000682 ***
## Minimum.Orbit.Intersection -1.170e+02  3.934e+00 -29.750  < 2e-16 ***
## Asc.Node.Longitude          -1.274e-03  5.281e-04  -2.413 0.015836 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 10835.2  on 21258  degrees of freedom
## Residual deviance:  2208.4  on 21253  degrees of freedom
## AIC: 2220.4
##
## Number of Fisher Scoring iterations: 10
```

```
kable(
  data.frame(Variable = sub("(Intercept)", "Intercept", names(coef(stepwise_hco_data)), fixed = TRUE),
             Coefficient = round(as.numeric(coef(stepwise_hco_data)), 6)),
  caption = "Variables Selected by Stepwise Selection (Bidirectional)",
  col.names = c("Variable Name", "Estimated Coefficient"),
  align = "l"
) %>% kable_styling(position = "left", full_width = FALSE)
```

Table 3: Variables Selected by Stepwise Selection (Bidirectional)

Variable Name	Estimated Coefficient
Intercept	69.568241
Absolute.Magnitude	-2.994389
Est.Dia.in.KM.max.	-5.107049
Orbit.Uncertainty	-0.069381
Minimum.Orbit.Intersection	-117.041167
Asc.Node.Longitude	-0.001274

4.6 Ridge Regression

```
train_hco_data$Hazardous      <- as.numeric(as.character(train_hco_data$Hazardous))
validation_hco_data$Hazardous <- as.numeric(as.character(validation_hco_data$Hazardous))
test_hco_data$Hazardous       <- as.numeric(as.character(test_hco_data$Hazardous))

predictors_train_hco_data      <- as.matrix(train_hco_data[, colnames(train_hco_data) != "Hazardous"])
target_train_hco_data          <- train_hco_data$Hazardous
predictors_validation_hco_data <- as.matrix(validation_hco_data[, names(validation_hco_data) != "Hazard
target_validation_hco_data     <- validation_hco_data$Hazardous

set.seed(5703)
ridge_5_fold_hco_data <- cv.glmnet(predictors_train_hco_data, target_train_hco_data,
                                   alpha = 0, family = "binomial", nfolds = 5)
ridge_lambda_min_hco_data <- ridge_5_fold_hco_data$lambda.min
```

```r
ridge_lambda_1se_hco_data <- ridge_5_fold_hco_data$lambda.1se

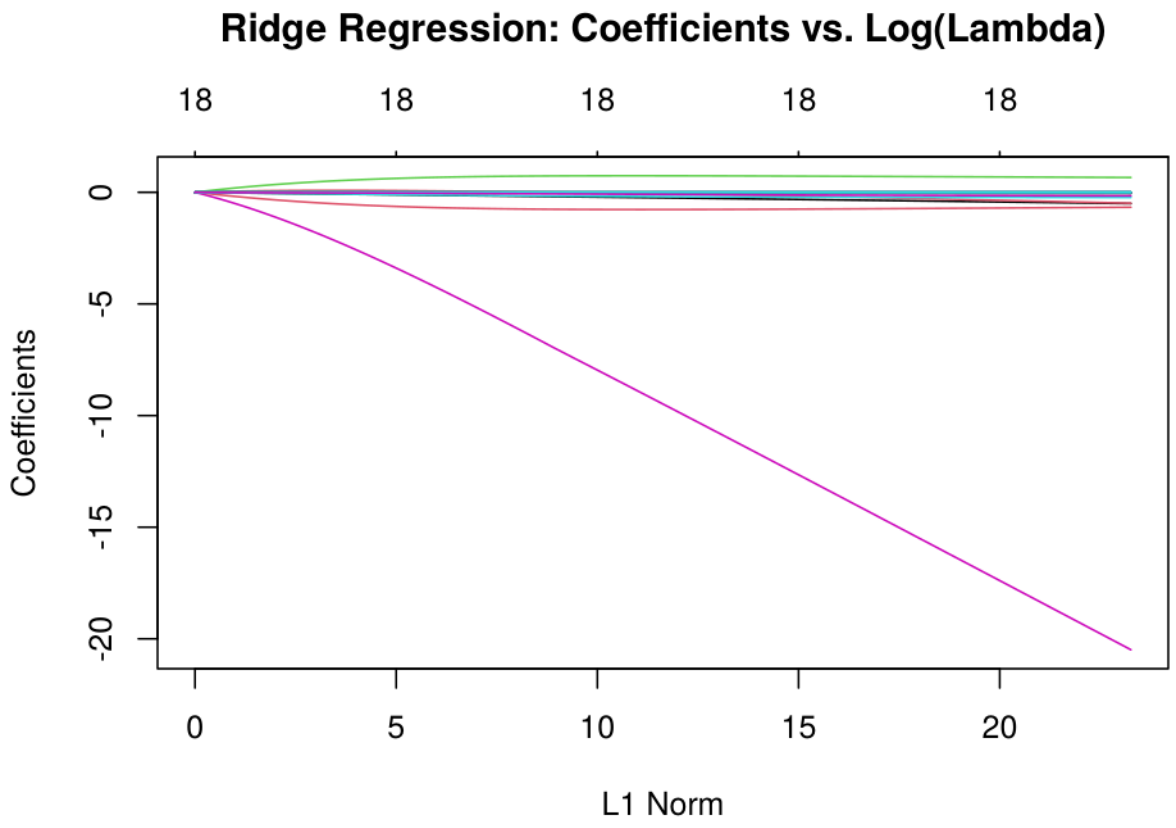
predict_response_ridge_min <- predict(ridge_5_fold_hco_data, newx = predictors_validation_hco_data,
                                      s = ridge_lambda_min_hco_data, type = "response")
predict_response_ridge_1se <- predict(ridge_5_fold_hco_data, newx = predictors_validation_hco_data,
                                      s = ridge_lambda_1se_hco_data, type = "response")
predict_class_ridge_min    <- predict(ridge_5_fold_hco_data, newx = predictors_validation_hco_data,
                                      s = ridge_lambda_min_hco_data, type = "class")

kable(
  data.frame(
    Metric = c("Lambda.min", "Lambda.1se", "Error (min)", "MSE (min)", "MSE (1se)"),
    Value  = c(round(ridge_lambda_min_hco_data, 6),
               round(ridge_lambda_1se_hco_data, 6),
               round(mean(predict_class_ridge_min != target_validation_hco_data), 6),
               round(mean((target_validation_hco_data - predict_response_ridge_min)^2), 6),
               round(mean((target_validation_hco_data - predict_response_ridge_1se)^2), 6))
  ),
  align = "c", caption = "Ridge Regression 5-fold Cross Validation"
) %>% kable_styling(position = "left", full_width = FALSE)
```

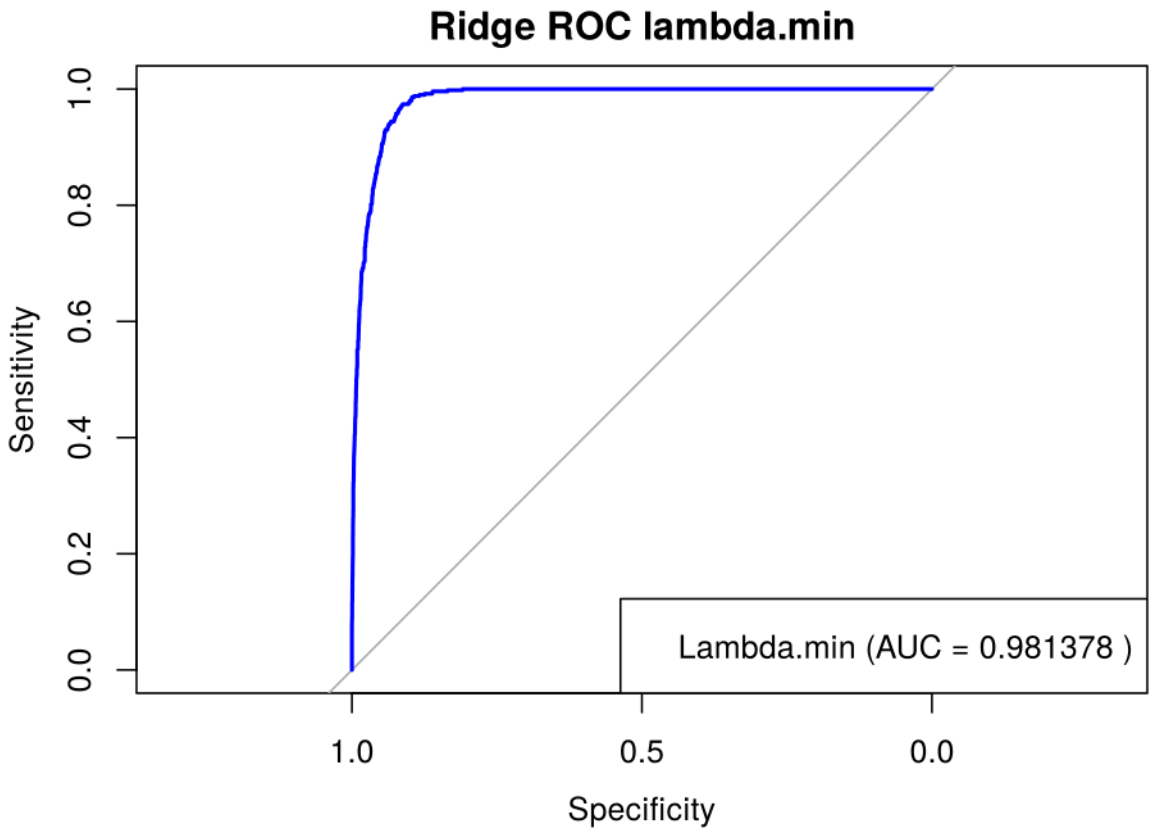
Table 4: Ridge Regression 5-fold Cross Validation

Metric	Value
Lambda.min	0.008233
Lambda.1se	0.009036
Error (min)	0.045724
MSE (min)	0.031115
MSE (1se)	0.031774

```r
ridge_fit_hco_data <- glmnet(predictors_train_hco_data, target_train_hco_data,
                             alpha = 0, family = "binomial")
plot(ridge_fit_hco_data)
title("Ridge Regression: Coefficients vs. Log(Lambda)", line = 3)
```




```
roc_lambda_min_ridge <- roc(target_validation_hco_data, predict_response_ridge_min[, 1], quiet = TRUE)
plot(roc_lambda_min_ridge, col = "blue", main = "Ridge ROC lambda.min")
legend("bottomright",
       legend = paste("Lambda.min (AUC =", round(auc(roc_lambda_min_ridge), 6), ")"))
```

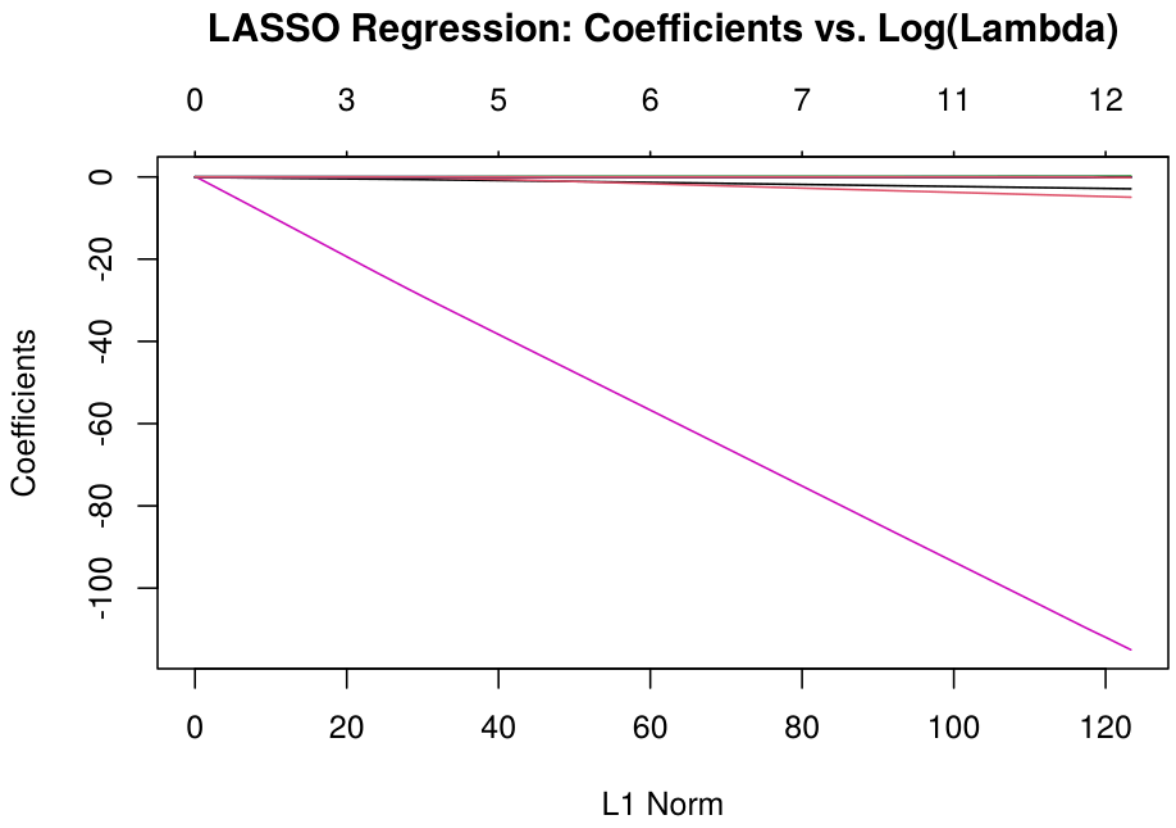


4.7 LASSO Regression

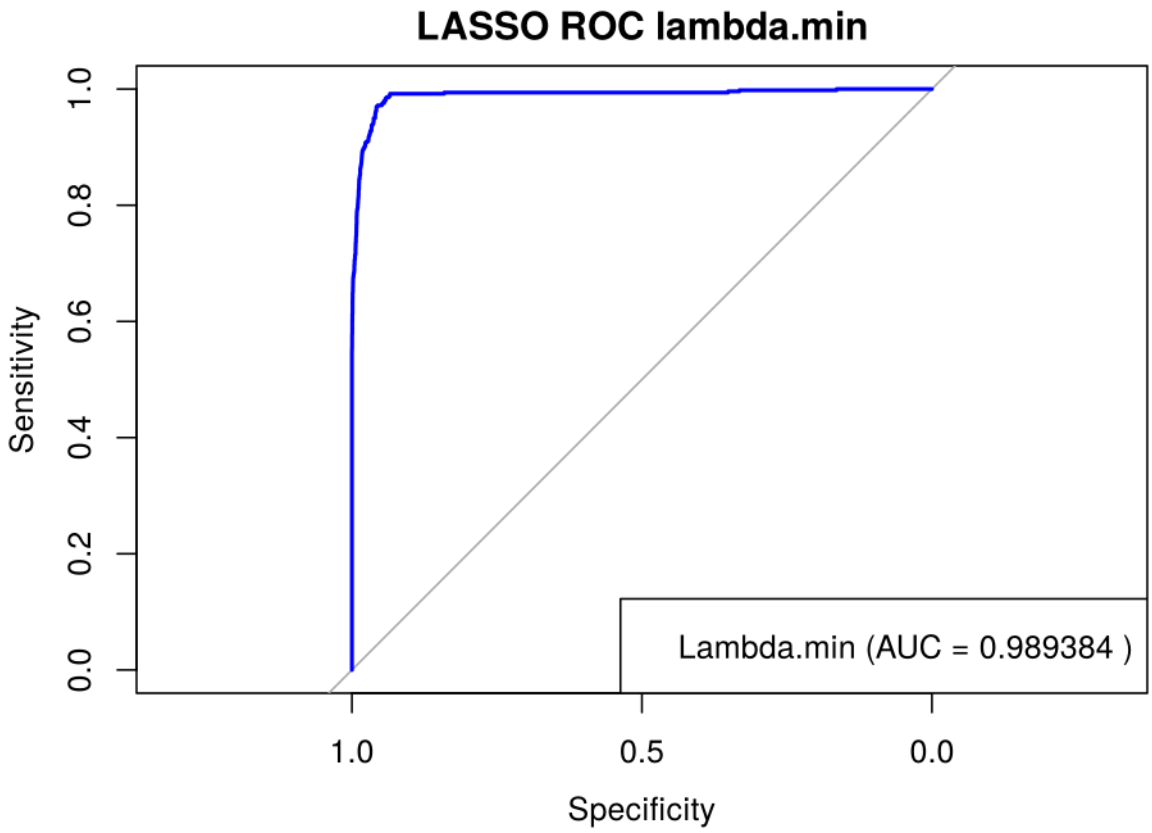
```
set.seed(5703)
lasso_5_fold_hco_data <- cv.glmnet(predictors_train_hco_data, target_train_hco_data,
                                   alpha = 1, family = "binomial", nfolds = 5)
lasso_lambda_min_hco_data <- lasso_5_fold_hco_data$lambda.min
lasso_lambda_1se_hco_data <- lasso_5_fold_hco_data$lambda.1se

predict_response_lasso_min <- predict(lasso_5_fold_hco_data, newx = predictors_validation_hco_data,
                                      s = lasso_lambda_min_hco_data, type = "response")

lasso_fit_hco_data <- glmnet(predictors_train_hco_data, target_train_hco_data,
                             alpha = 1, family = "binomial")
plot(lasso_fit_hco_data)
title("LASSO Regression: Coefficients vs. Log(Lambda)", line = 3)
```




```r
roc_lambda_min_lasso <- roc(target_validation_hco_data, predict_response_lasso_min[, 1], quiet = TRUE)
plot(roc_lambda_min_lasso, col = "blue", main = "LASSO ROC lambda.min")
legend("bottomright",
       legend = paste("Lambda.min (AUC =", round(auc(roc_lambda_min_lasso), 6), ")"))
```



4.8 Step 2 Model Comparison (Validation Set)

```
val_hazardous_numeric <- as.numeric(as.character(validation_hco_data$Hazardous))

best_vars <- names(coef(best_subset_hco_data, best_model_bic))
best_vars <- best_vars[best_vars != "(Intercept)"]
form_best <- as.formula(paste("Hazardous ~", paste(best_vars, collapse = " + ")))
fit_best  <- glm(form_best, data = train_hco_data, family = binomial)

pred_baseline <- predict(baseline_model_hco_data, newdata = validation_hco_data, type = "response")
pred_stepwise <- predict(stepwise_hco_data,       newdata = validation_hco_data, type = "response")
pred_best     <- predict(fit_best,                newdata = validation_hco_data, type = "response")
pred_ridge    <- predict(ridge_5_fold_hco_data, newx = predictors_validation_hco_data,
                         s = ridge_lambda_min_hco_data, type = "response")[, 1]
pred_lasso    <- predict(lasso_5_fold_hco_data, newx = predictors_validation_hco_data,
                         s = lasso_lambda_min_hco_data, type = "response")[, 1]

roc_baseline <- roc(val_hazardous_numeric, pred_baseline, quiet = TRUE)
roc_stepwise <- roc(val_hazardous_numeric, pred_stepwise, quiet = TRUE)
roc_best     <- roc(val_hazardous_numeric, pred_best,     quiet = TRUE)
roc_ridge    <- roc(val_hazardous_numeric, pred_ridge,    quiet = TRUE)
roc_lasso    <- roc(val_hazardous_numeric, pred_lasso,    quiet = TRUE)

step2_table <- data.frame(
  Method = c("Baseline (Full Logistic)", "Stepwise (Bidirectional)",
```

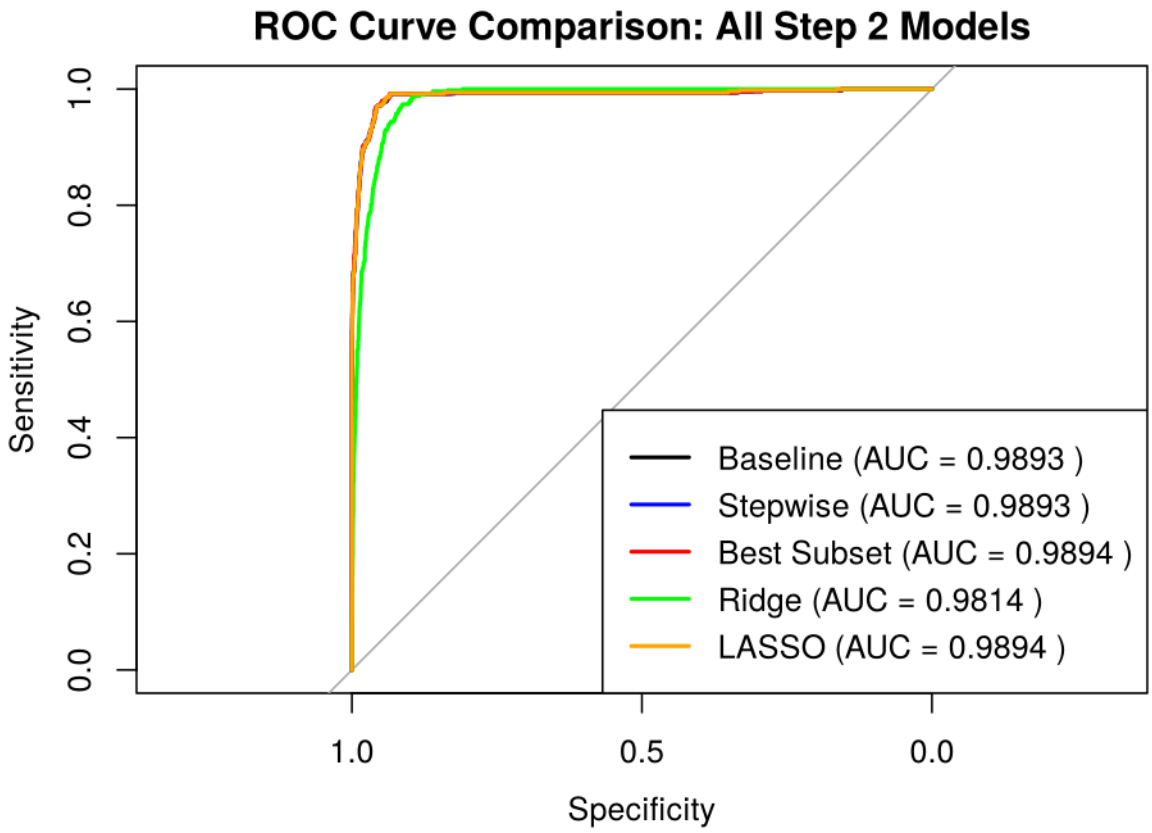
```
             "Best Subset", "Ridge (lambda.min)", "LASSO (lambda.min)"),
  Variables = c(length(coef(baseline_model_hco_data)) - 1,
                length(coef(stepwise_hco_data)) - 1,
                best_model_bic,
                sum(coef(ridge_5_fold_hco_data, s = ridge_lambda_min_hco_data) != 0) - 1,
                sum(coef(lasso_5_fold_hco_data, s = lasso_lambda_min_hco_data) != 0) - 1),
  AIC = c(round(AIC(baseline_model_hco_data), 4), round(AIC(stepwise_hco_data), 4),
          round(AIC(fit_best), 4), NA, NA),
  AUC = round(c(as.numeric(auc(roc_baseline)), as.numeric(auc(roc_stepwise)),
                as.numeric(auc(roc_best)), as.numeric(auc(roc_ridge)),
                as.numeric(auc(roc_lasso))), 4)
)

kable(step2_table,
      caption = "Final Model Comparison Across All Step 2 Methods (Validation Set)",
      col.names = c("Method", "Variables Used", "AIC (Training)", "AUC (Validation)"),
      align = "c") %>%
  kable_styling(position = "left", full_width = FALSE)
```

Table 5: Final Model Comparison Across All Step 2 Methods (Validation Set)

Method	Variables Used	AIC (Training)	AUC (Validation)
Baseline (Full Logistic)	18	2240.344	0.9893
Stepwise (Bidirectional)	5	2220.360	0.9893
Best Subset	6	2225.745	0.9894
Ridge (lambda.min)	18	NA	0.9814
LASSO (lambda.min)	13	NA	0.9894

```
plot(roc_baseline, col = "black", lwd = 2, main = "ROC Curve Comparison: All Step 2 Models")
lines(roc_stepwise, col = "blue",   lwd = 2)
lines(roc_best,     col = "red",    lwd = 2)
lines(roc_ridge,    col = "green",  lwd = 2)
lines(roc_lasso,    col = "orange", lwd = 2)
legend("bottomright",
       legend = c(paste("Baseline (AUC =",    round(as.numeric(auc(roc_baseline)), 4), ")"),
                  paste("Stepwise (AUC =",    round(as.numeric(auc(roc_stepwise)), 4), ")"),
                  paste("Best Subset (AUC =", round(as.numeric(auc(roc_best)), 4), ")"),
                  paste("Ridge (AUC =",       round(as.numeric(auc(roc_ridge)), 4), ")"),
                  paste("LASSO (AUC =",       round(as.numeric(auc(roc_lasso)), 4), ")")),
       col = c("black", "blue", "red", "green", "orange"), lwd = 2)
```



5 Ensemble Methods: Random Forest, Boosting, and Bagging

Each forest draws an equal number of hazardous and non-hazardous objects per tree (`sampsize`), which balances the classes during training only. The validation and test sets keep the real-world 93/7 class ratio.

5.1 Random Forest

5.1.1 Model 1: Baseline Random Forest

```
set.seed(5703)
comet_rf_model <- randomForest(
  Hazardous ~ ., data = comet_train, ntree = 500,
  mtry = floor(sqrt(ncol(comet_train) - 1)),
  sampsize = c("FALSE" = minority_n, "TRUE" = minority_n),
  importance = TRUE
)
comet_preds <- predict(comet_rf_model, newdata = comet_val)
confusionMatrix(comet_preds, comet_val$Hazardous, positive = "TRUE")
```

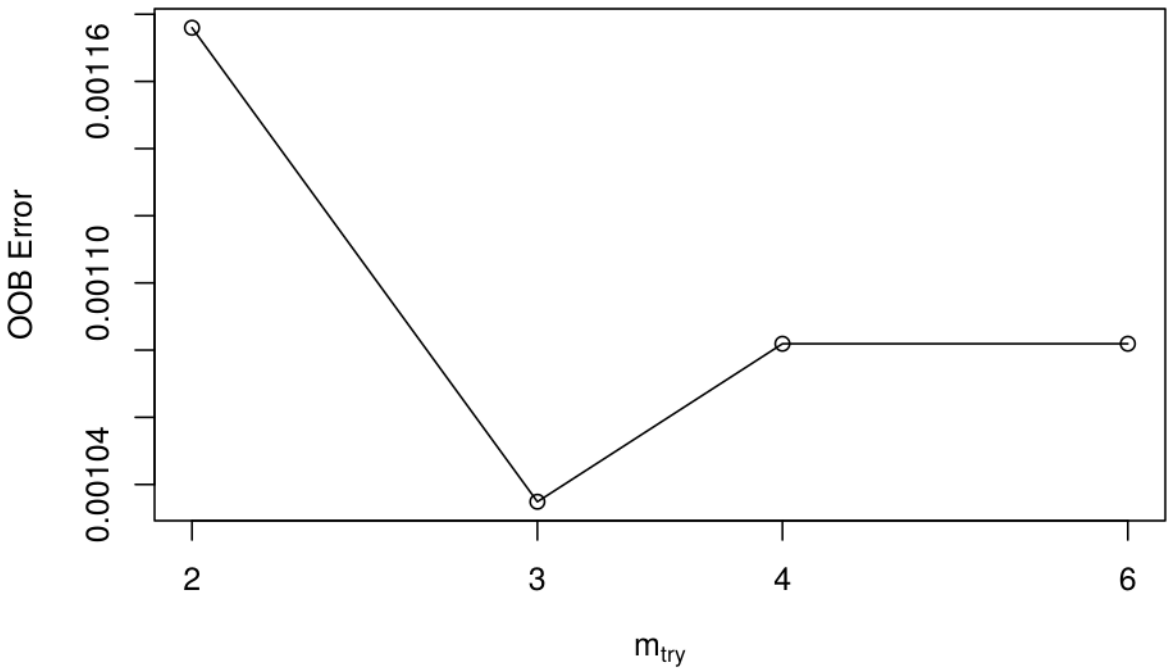
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6575    2
##      TRUE     12  497
```

```
##
##                Accuracy : 0.998
##                  95% CI : (0.9967, 0.9989)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.985
##
##  Mcnemar's Test P-Value : 0.01616
##
##             Sensitivity : 0.99599
##             Specificity : 0.99818
##          Pos Pred Value : 0.97642
##          Neg Pred Value : 0.99970
##              Prevalence : 0.07042
##          Detection Rate : 0.07014
##    Detection Prevalence : 0.07183
##       Balanced Accuracy : 0.99709
##
##        'Positive' Class : TRUE
##
```

5.1.2 mtry Tuning via OOB Error

```r
set.seed(5703)
comet_tuned <- tuneRF(
  x = comet_train[, -which(names(comet_train) == "Hazardous")],
  y = comet_train$Hazardous,
  stepFactor = 1.5, improve = 0.01, ntreeTry = 300, trace = TRUE, plot = TRUE
)
```

```
## mtry = 4  OOB error = 0.11%
## Searching left ...
## mtry = 3     OOB error = 0.1%
## 0.04347826 0.01
## mtry = 2     OOB error = 0.12%
## -0.1363636 0.01
## Searching right ...
## mtry = 6     OOB error = 0.11%
## -0.04545455 0.01
```



5.1.3 Model 2: Tuned mtry with Class Weights

```
set.seed(5703)
comet_rf_model_tune <- randomForest(
  Hazardous ~ ., data = comet_train, ntree = 500, mtry = 4,
  sampsize = c("FALSE" = minority_n, "TRUE" = minority_n),
  classwt = c("FALSE" = 1, "TRUE" = 10),
  importance = TRUE
)
comet_preds_1 <- predict(comet_rf_model_tune, newdata = comet_val)
confusionMatrix(comet_preds_1, comet_val$Hazardous, positive = "TRUE")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6568    1
##      TRUE     19  498
##
##                Accuracy : 0.9972
##                  95% CI : (0.9956, 0.9983)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9788
##
```

```
##  Mcnemar's Test P-Value : 0.0001439
##
##             Sensitivity : 0.99800
##             Specificity : 0.99712
##          Pos Pred Value : 0.96325
##          Neg Pred Value : 0.99985
##              Prevalence : 0.07042
##          Detection Rate : 0.07028
##    Detection Prevalence : 0.07296
##       Balanced Accuracy : 0.99756
##
##        'Positive' Class : TRUE
##
```

5.1.4 Model 3: Lowered Classification Threshold

```
set.seed(5703)
comet_rf_model_tune_1 <- randomForest(
  Hazardous ~ ., data = comet_train, ntree = 500, mtry = 4,
  sampsize = c("FALSE" = minority_n, "TRUE" = minority_n),
  importance = TRUE
)
pred_probs_1 <- predict(comet_rf_model_tune_1, newdata = as.data.frame(comet_val), type = "prob")
pred_class_1 <- factor(ifelse(pred_probs_1[, "TRUE"] >= 0.15, "TRUE", "FALSE"),
                       levels = c("FALSE", "TRUE"))
confusionMatrix(pred_class_1, comet_val$Hazardous, positive = "TRUE")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6507    0
##      TRUE     80  499
##
##                Accuracy : 0.9887
##                  95% CI : (0.986, 0.991)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9197
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 1.00000
##             Specificity : 0.98785
##          Pos Pred Value : 0.86183
##          Neg Pred Value : 1.00000
##              Prevalence : 0.07042
##          Detection Rate : 0.07042
##    Detection Prevalence : 0.08171
##       Balanced Accuracy : 0.99393
##
##        'Positive' Class : TRUE
##
```

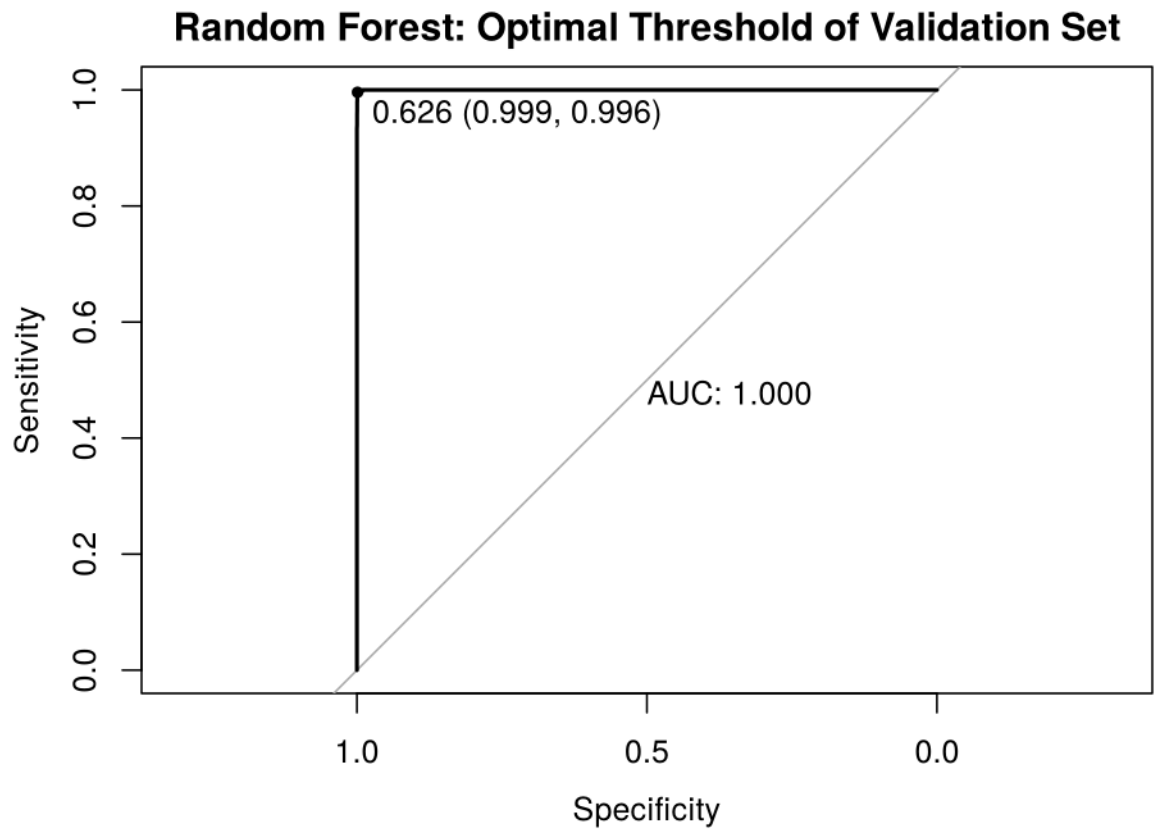
5.1.5 Model 4: ROC-Optimized Threshold (Final RF Model)

Rather than guessing a cutoff, the threshold is read directly from the validation ROC curve: the point that best balances sensitivity and specificity (Youden's index). The same method is used for boosting below.

```
set.seed(5703)
comet_rf_model_tune_2 <- randomForest(
  Hazardous ~ ., data = comet_train, ntree = 500, mtry = 4,
  sampsize = c("FALSE" = minority_n, "TRUE" = minority_n),
  importance = TRUE
)

pred_probs_2 <- predict(comet_rf_model_tune_2, newdata = as.data.frame(comet_val), type = "prob")

roc_rf_val   <- roc(comet_val$Hazardous, pred_probs_2[, "TRUE"], quiet = TRUE)
rf_threshold <- coords(roc_rf_val, "best", ret = "threshold", transpose = FALSE)$threshold[1]
plot(roc_rf_val, print.thres = "best", print.auc = TRUE,
     main = "Random Forest: Optimal Threshold of Validation Set")
```




```
cat("Optimal Threshold Cutoff:", rf_threshold, "\n")
```

```
## Optimal Threshold Cutoff: 0.626
```

```
pred_class_2 <- factor(ifelse(pred_probs_2[, "TRUE"] >= rf_threshold, "TRUE", "FALSE"),
                       levels = c("FALSE", "TRUE"))
confusionMatrix(pred_class_2, comet_val$Hazardous, positive = "TRUE")
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6580    2
##      TRUE      7  497
##
##                Accuracy : 0.9987
##                  95% CI : (0.9976, 0.9994)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9903
##
##  Mcnemar's Test P-Value : 0.1824
##
##             Sensitivity : 0.99599
##             Specificity : 0.99894
##          Pos Pred Value : 0.98611
##          Neg Pred Value : 0.99970
##              Prevalence : 0.07042
##          Detection Rate : 0.07014
##    Detection Prevalence : 0.07113
##       Balanced Accuracy : 0.99746
##
##        'Positive' Class : TRUE
##
```

5.1.6 Final RF Test Set Evaluation

```
pred_probs_3 <- predict(comet_rf_model_tune_2, newdata = as.data.frame(comet_test), type = "prob")
pred_class_3 <- factor(ifelse(pred_probs_3[, "TRUE"] >= rf_threshold, "TRUE", "FALSE"),
                       levels = c("FALSE", "TRUE"))
confusionMatrix(pred_class_3, comet_test$Hazardous, positive = "TRUE")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6581    8
##      TRUE      5  491
##
##                Accuracy : 0.9982
##                  95% CI : (0.9969, 0.999)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9859
##
##  Mcnemar's Test P-Value : 0.5791
##
##             Sensitivity : 0.98397
##             Specificity : 0.99924
##          Pos Pred Value : 0.98992
##          Neg Pred Value : 0.99879
```

```
##              Prevalence : 0.07043
##          Detection Rate : 0.06930
##    Detection Prevalence : 0.07001
##       Balanced Accuracy : 0.99160
##
##        'Positive' Class : TRUE
##
```

5.2 Gradient Boosting

5.2.1 Data Recoding for Boosting Compatibility

caret needs class labels that are valid R names, so TRUE/FALSE become Yes/No.

```
recode_yes_no <- function(d) {
  d$Hazardous <- factor(ifelse(d$Hazardous == "TRUE", "Yes", "No"), levels = c("No", "Yes"))
  d
}
comet_train_boost <- recode_yes_no(comet_train)
comet_val_boost   <- recode_yes_no(comet_val)
comet_test_boost  <- recode_yes_no(comet_test)
```

5.2.2 Model Training: 5-Fold CV with Grid Search

Down-sampling is applied inside each training fold only, and the list of seeds makes the grid search reproducible.

```
set.seed(5703)
gbm_grid <- expand.grid(interaction.depth = 1:5,
                        n.trees = seq(100, 600, by = 100),
                        shrinkage = c(0.001, 0.01, 0.1),
                        n.minobsinnode = c(10, 20))

n_models <- nrow(gbm_grid)
setSeeds <- vector(mode = "list", length = 6)
for (i in 1:5) setSeeds[[i]] <- sample.int(100000, n_models)
setSeeds[[6]] <- sample.int(100000, 1)

fitControl <- trainControl(method = "cv", number = 5, classProbs = TRUE,
                           summaryFunction = twoClassSummary,
                           sampling = "down", seeds = setSeeds)

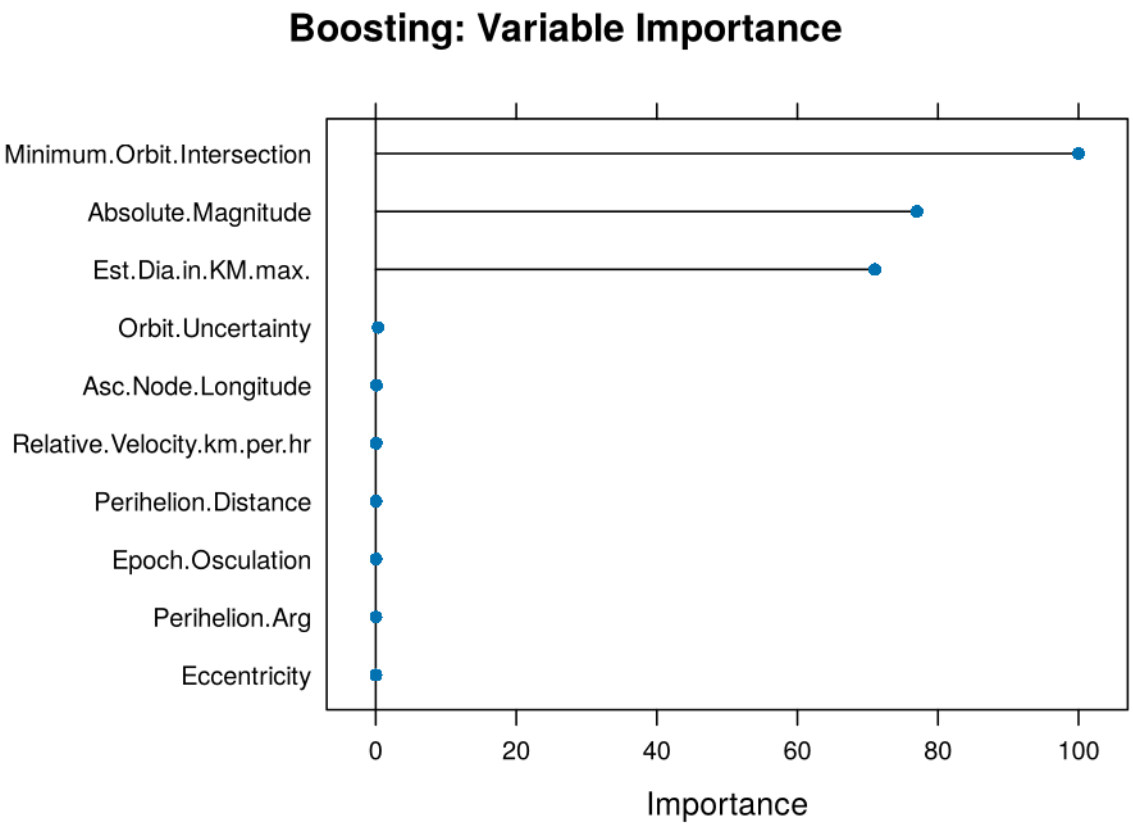
model_boost <- train(Hazardous ~ ., data = comet_train_boost,
                     method = "gbm", distribution = "bernoulli",
                     trControl = fitControl, tuneGrid = gbm_grid,
                     metric = "ROC", verbose = FALSE)
```

5.2.3 Optimal Tuning and Variable Importance

```
model_boost$bestTune
```

```
##     n.trees interaction.depth shrinkage n.minobsinnode
## 119     500                 5      0.01             20
```

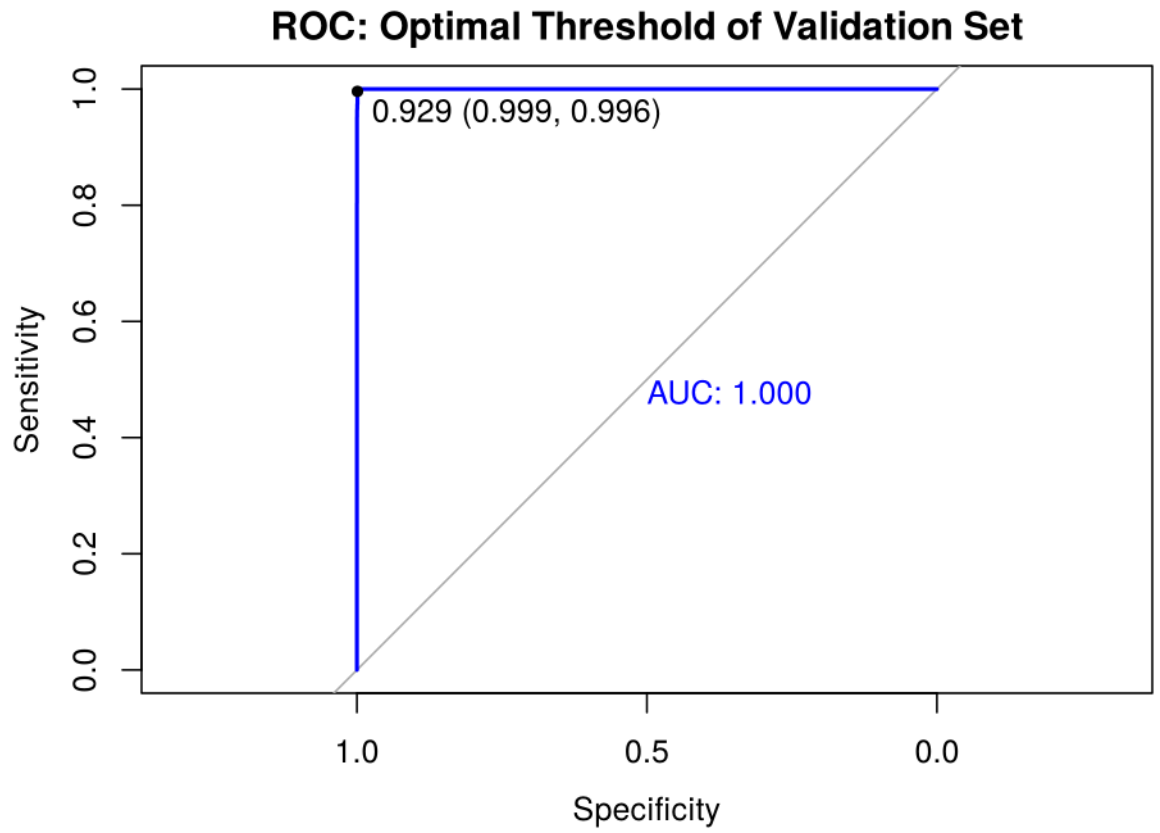
```
plot(varImp(model_boost), top = 10, main = "Boosting: Variable Importance")
```



5.2.4 Validation Set ROC: Threshold Selection

Because the model trained on 50/50 down-sampled folds, its predicted probabilities run high for a dataset where only 7% of objects are hazardous. That is why the ROC-optimal cutoff lands well above 0.5.

```
probs_boost_val <- predict(model_boost, newdata = comet_val_boost, type = "prob")[, "Yes"]
roc_boost_val   <- roc(response = comet_val_boost$Hazardous, predictor = probs_boost_val, quiet = TRUE)
plot(roc_boost_val, print.thres = "best", print.auc = TRUE, col = "blue",
     main = "ROC: Optimal Threshold of Validation Set")
```




```r
optimal   <- coords(roc_boost_val, "best", ret = c("threshold", "specificity", "sensitivity"),
                    transpose = FALSE)
threshold <- optimal$threshold[1]
cat("Optimal Threshold Cutoff:", threshold, "\n")
```

```
## Optimal Threshold Cutoff: 0.9293955
```

5.2.5 Validation Set Evaluation

```r
preds_boost_val <- factor(ifelse(probs_boost_val >= threshold, "Yes", "No"), levels = c("No", "Yes"))
cm_boost_val    <- confusionMatrix(preds_boost_val, comet_val_boost$Hazardous, positive = "Yes")
cm_boost_val
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  6581    2
##        Yes    6  497
##
##                Accuracy : 0.9989
##                  95% CI : (0.9978, 0.9995)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9914
```

```
##
##  Mcnemar's Test P-Value : 0.2888
##
##             Sensitivity : 0.99599
##             Specificity : 0.99909
##          Pos Pred Value : 0.98807
##          Neg Pred Value : 0.99970
##              Prevalence : 0.07042
##          Detection Rate : 0.07014
##    Detection Prevalence : 0.07099
##       Balanced Accuracy : 0.99754
##
##        'Positive' Class : Yes
##
```

```
cat("ROC-AUC:", round(auc(roc_boost_val), 4), "\n")
```

```
## ROC-AUC: 0.9999
```

5.2.6 Test Set Evaluation

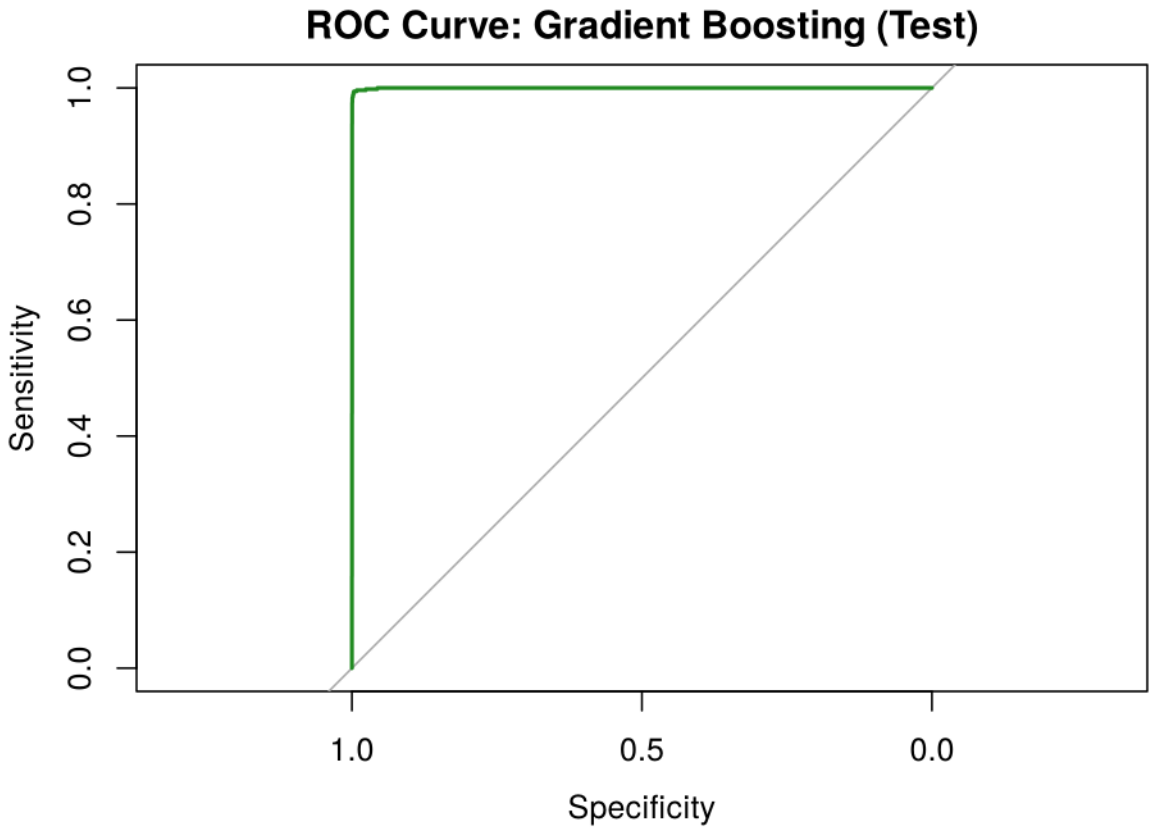
```
probs_boost_test <- predict(model_boost, newdata = comet_test_boost, type = "prob")[, "Yes"]
preds_boost_test <- factor(ifelse(probs_boost_test >= threshold, "Yes", "No"), levels = c("No", "Yes"))
cm_boost_test    <- confusionMatrix(preds_boost_test, comet_test_boost$Hazardous, positive = "Yes")
cm_boost_test
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   No  Yes
##        No  6580    8
##        Yes    6  491
##
##                Accuracy : 0.998
##                  95% CI : (0.9967, 0.9989)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.9849
##
##  Mcnemar's Test P-Value : 0.7893
##
##             Sensitivity : 0.98397
##             Specificity : 0.99909
##          Pos Pred Value : 0.98793
##          Neg Pred Value : 0.99879
##              Prevalence : 0.07043
##          Detection Rate : 0.06930
##    Detection Prevalence : 0.07015
##       Balanced Accuracy : 0.99153
##
##        'Positive' Class : Yes
##
```

```
roc_boost <- roc(response = comet_test_boost$Hazardous, predictor = probs_boost_test,
                 levels = c("No", "Yes"), direction = "<", quiet = TRUE)
ROC_AUC <- round(auc(roc_boost), 4)
cat("ROC-AUC:", ROC_AUC, "\n")
```

```
## ROC-AUC: 0.9996
```

```
plot(roc_boost, col = "forestgreen", main = "ROC Curve: Gradient Boosting (Test)", lwd = 2)
```



5.3 Bagging

Bagging is a random forest that considers every predictor at each split (`mtry = p`).

5.3.1 Model Fitting: 100, 300, 500 Trees

```
p <- ncol(comet_train) - 1

set.seed(5703)
bag_100 <- randomForest(Hazardous ~ ., data = comet_train, mtry = p, ntree = 100,
                        sampsize = c("FALSE" = minority_n, "TRUE" = minority_n), importance = TRUE)
set.seed(5703)
bag_300 <- randomForest(Hazardous ~ ., data = comet_train, mtry = p, ntree = 300,
                        sampsize = c("FALSE" = minority_n, "TRUE" = minority_n), importance = TRUE)
set.seed(5703)
bag_500 <- randomForest(Hazardous ~ ., data = comet_train, mtry = p, ntree = 500,
                        sampsize = c("FALSE" = minority_n, "TRUE" = minority_n), importance = TRUE)
```

```
bag_100
```

```
##
## Call:
##  randomForest(formula = Hazardous ~ ., data = comet_train, mtry = p,      ntree = 100, sampsize = c(
##                Type of random forest: classification
##                      Number of trees: 100
## No. of variables tried at each split: 18
##
##         OOB estimate of  error rate: 0.19%
## Confusion matrix:
##       FALSE TRUE class.error
## FALSE 19732   29 0.001467537
## TRUE     11 1487 0.007343124
```

```
bag_300
```

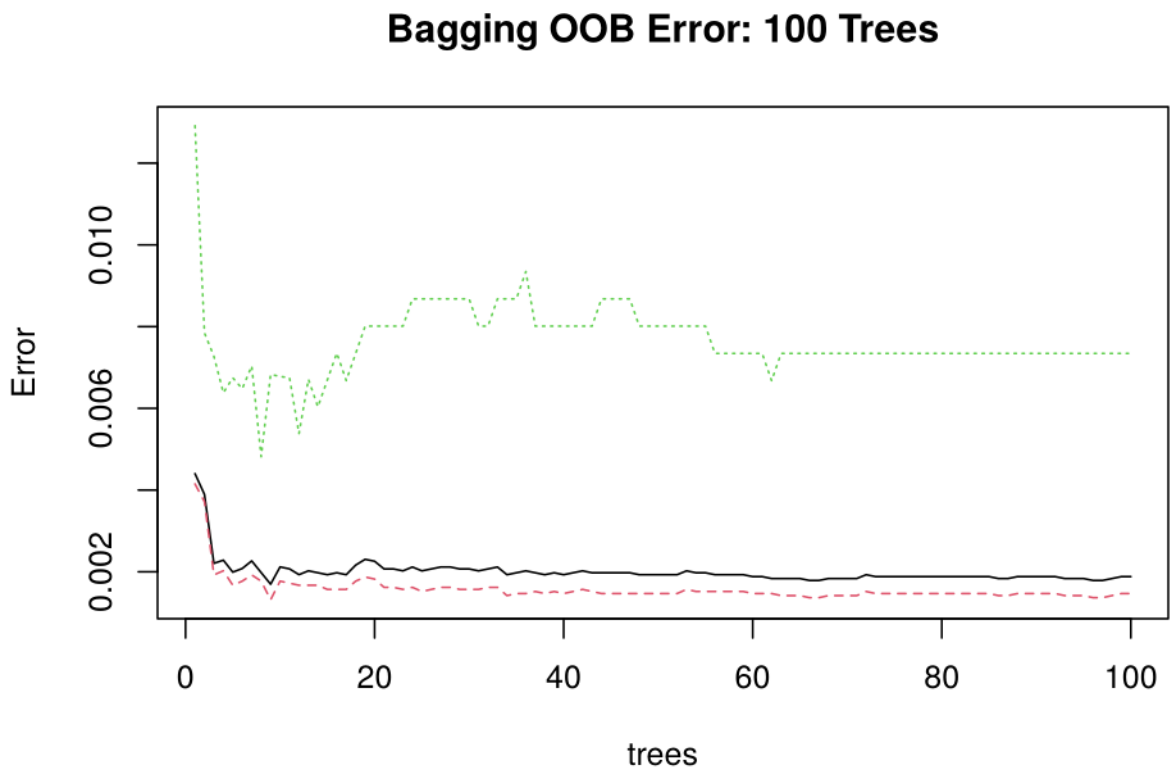
```
##
## Call:
##  randomForest(formula = Hazardous ~ ., data = comet_train, mtry = p,      ntree = 300, sampsize = c(
##                Type of random forest: classification
##                      Number of trees: 300
## No. of variables tried at each split: 18
##
##         OOB estimate of  error rate: 0.18%
## Confusion matrix:
##       FALSE TRUE class.error
## FALSE 19734   27 0.001366328
## TRUE     11 1487 0.007343124
```

```
bag_500
```

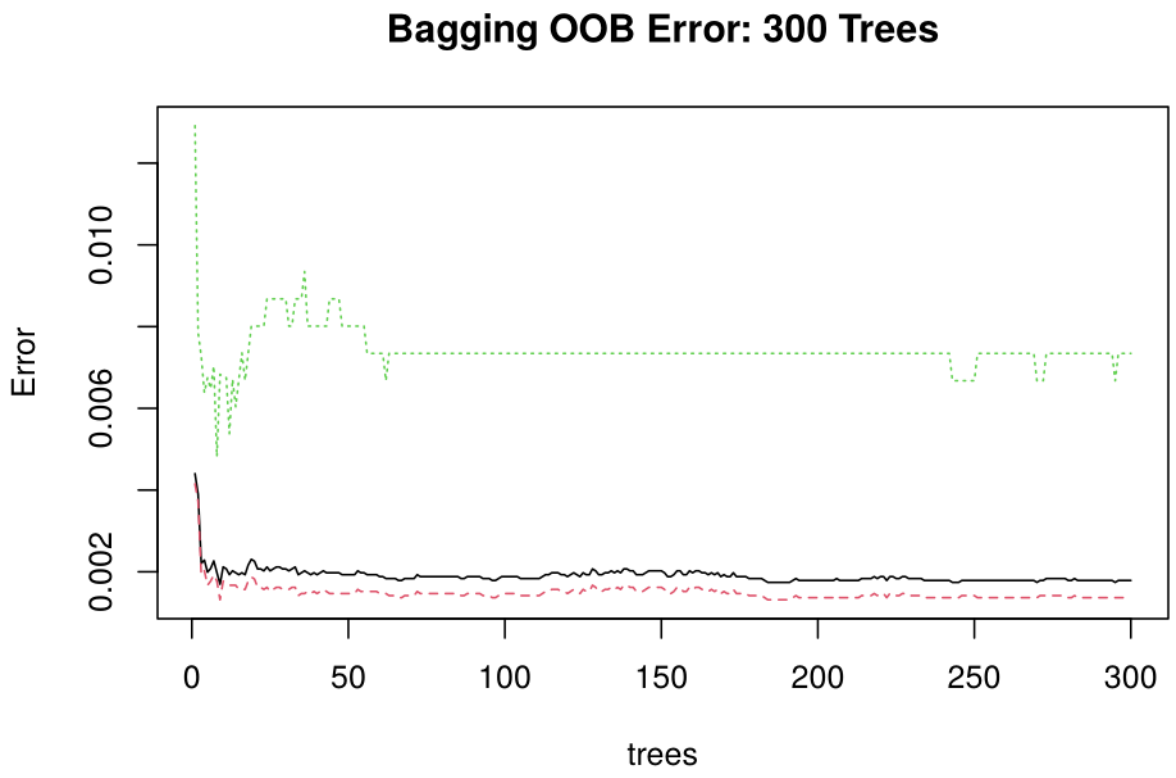
```
##
## Call:
##  randomForest(formula = Hazardous ~ ., data = comet_train, mtry = p,      ntree = 500, sampsize = c(
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 18
##
##         OOB estimate of  error rate: 0.17%
## Confusion matrix:
##       FALSE TRUE class.error
## FALSE 19735   26 0.001315723
## TRUE     11 1487 0.007343124
```

5.3.2 OOB Error Plots

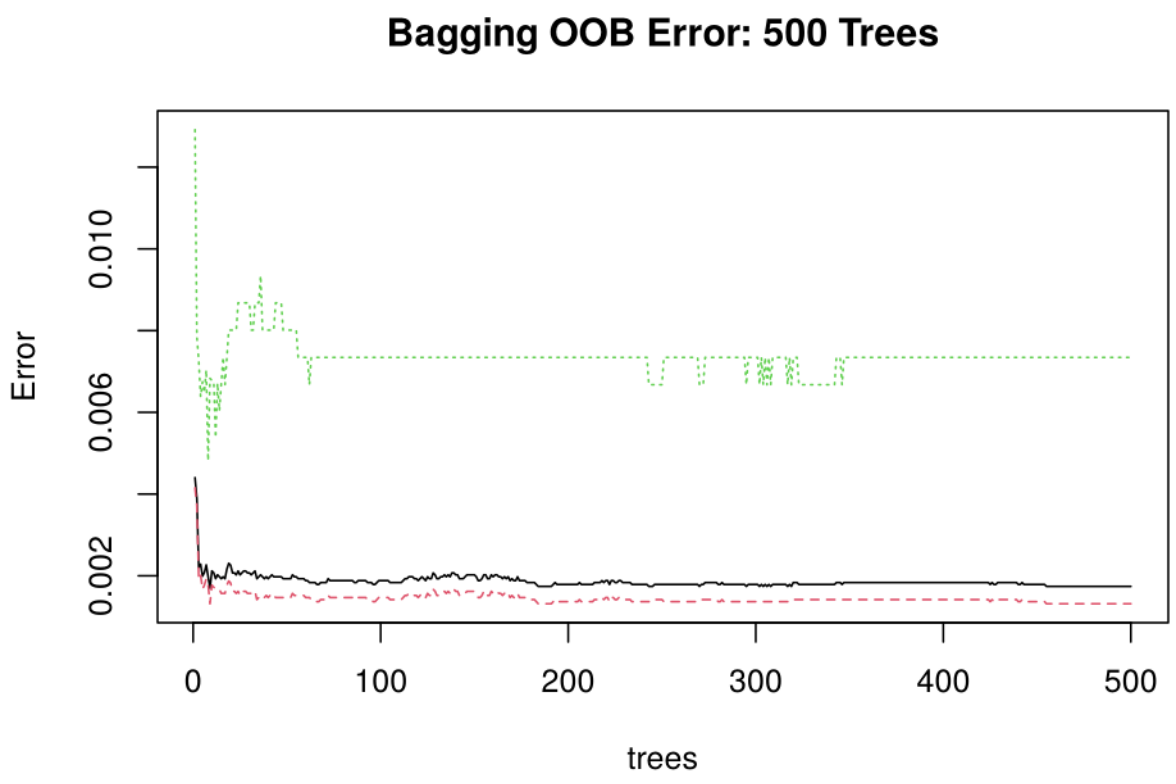
```
plot(bag_100, main = "Bagging OOB Error: 100 Trees")
```




```
plot(bag_300, main = "Bagging OOB Error: 300 Trees")
```




```
plot(bag_500, main = "Bagging OOB Error: 500 Trees")
```



5.3.3 Validation Set Comparison

```r
get_classification_metrics <- function(model, newdata, truth_col = "Hazardous",
                                       positive_class = "TRUE", threshold = 0.5) {
  pred_prob  <- predict(model, newdata = newdata, type = "prob")[, positive_class]
  pred_class <- factor(ifelse(pred_prob >= threshold, "TRUE", "FALSE"), levels = c("FALSE", "TRUE"))
  cm <- confusionMatrix(pred_class, newdata[[truth_col]], positive = positive_class)
  roc_obj <- roc(response = newdata[[truth_col]], predictor = pred_prob,
                 levels = c("FALSE", "TRUE"), direction = "<", quiet = TRUE)
  data.frame(
    Accuracy    = as.numeric(cm$overall["Accuracy"]),
    Precision   = as.numeric(cm$byClass["Pos Pred Value"]),
    Recall      = as.numeric(cm$byClass["Sensitivity"]),
    Specificity = as.numeric(cm$byClass["Specificity"]),
    F1          = as.numeric(cm$byClass["F1"]),
    AUC         = as.numeric(auc(roc_obj))
  )
}

# A deliberately low threshold: an object is flagged if even 2.5% of trees vote
# "hazardous". This prioritizes recall (missing a hazardous object is the costly
# error) at the price of more false alarms. It is chosen on validation and
# reused unchanged on the test set.
bag_threshold <- 0.025
```

```r
validation_results <- rbind(
  cbind(Model = "Bagging_100", get_classification_metrics(bag_100, comet_val, threshold = bag_threshold
  cbind(Model = "Bagging_300", get_classification_metrics(bag_300, comet_val, threshold = bag_threshold
  cbind(Model = "Bagging_500", get_classification_metrics(bag_500, comet_val, threshold = bag_threshold
)
kable(validation_results, caption = "Bagging Validation Set Results", align = "c", digits = 4) %>%
  kable_styling(position = "left", full_width = FALSE)
```

Table 6: Bagging Validation Set Results

Model	Accuracy	Precision	Recall	Specificity	F1	AUC
Bagging_100	0.9664	0.6771	1	0.9639	0.8074	0.9997
Bagging_300	0.9660	0.6743	1	0.9634	0.8055	0.9999
Bagging_500	0.9660	0.6743	1	0.9634	0.8055	0.9999

5.3.4 Final Bagging Model: Test Set Evaluation

The same threshold chosen on the validation set is applied to the test set.

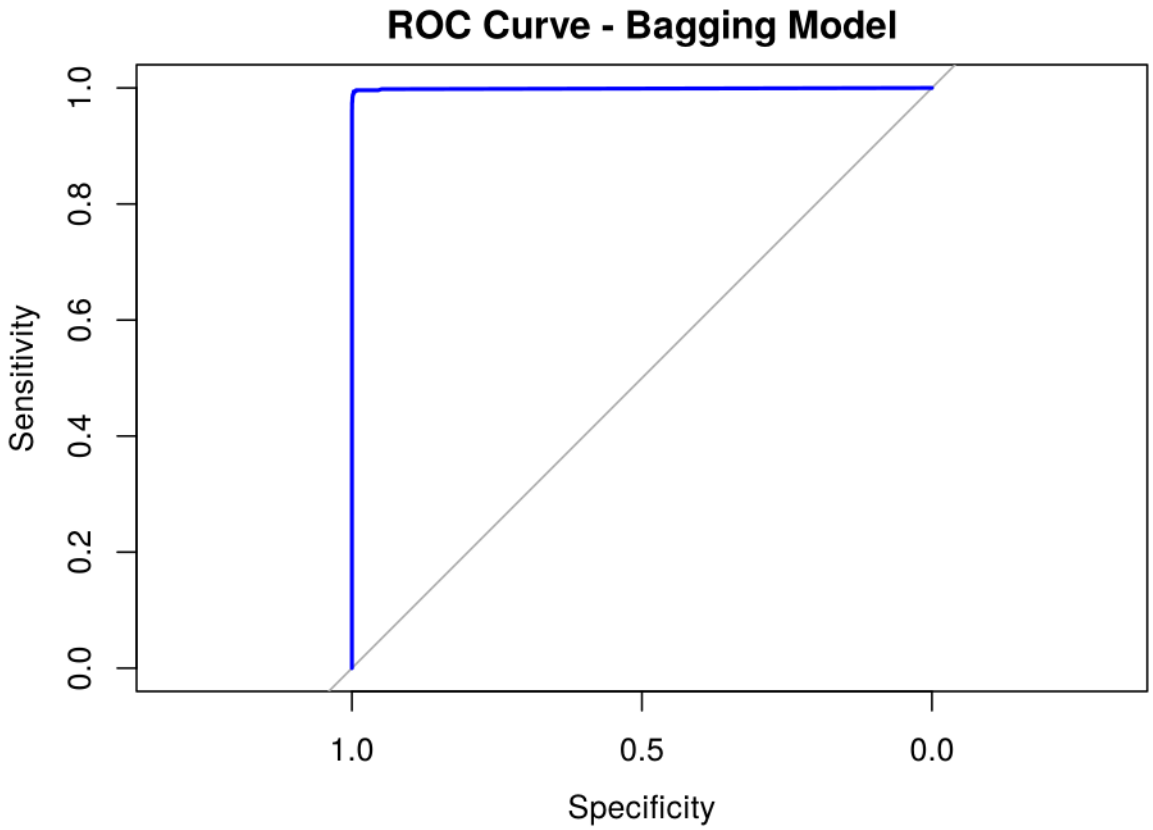
```r
final_bag      <- bag_300
bag_pred_prob  <- predict(final_bag, newdata = comet_test, type = "prob")[, "TRUE"]
bag_pred_class <- factor(ifelse(bag_pred_prob >= bag_threshold, "TRUE", "FALSE"),
                         levels = c("FALSE", "TRUE"))
bag_cm <- confusionMatrix(bag_pred_class, comet_test$Hazardous, positive = "TRUE")
bag_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction FALSE TRUE
##      FALSE  6350    2
##      TRUE    236  497
##
##                Accuracy : 0.9664
##                  95% CI : (0.9619, 0.9705)
##     No Information Rate : 0.9296
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7891
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.99599
##             Specificity : 0.96417
##          Pos Pred Value : 0.67804
##          Neg Pred Value : 0.99969
##              Prevalence : 0.07043
##          Detection Rate : 0.07015
##    Detection Prevalence : 0.10346
##       Balanced Accuracy : 0.98008
##
##        'Positive' Class : TRUE
##
```

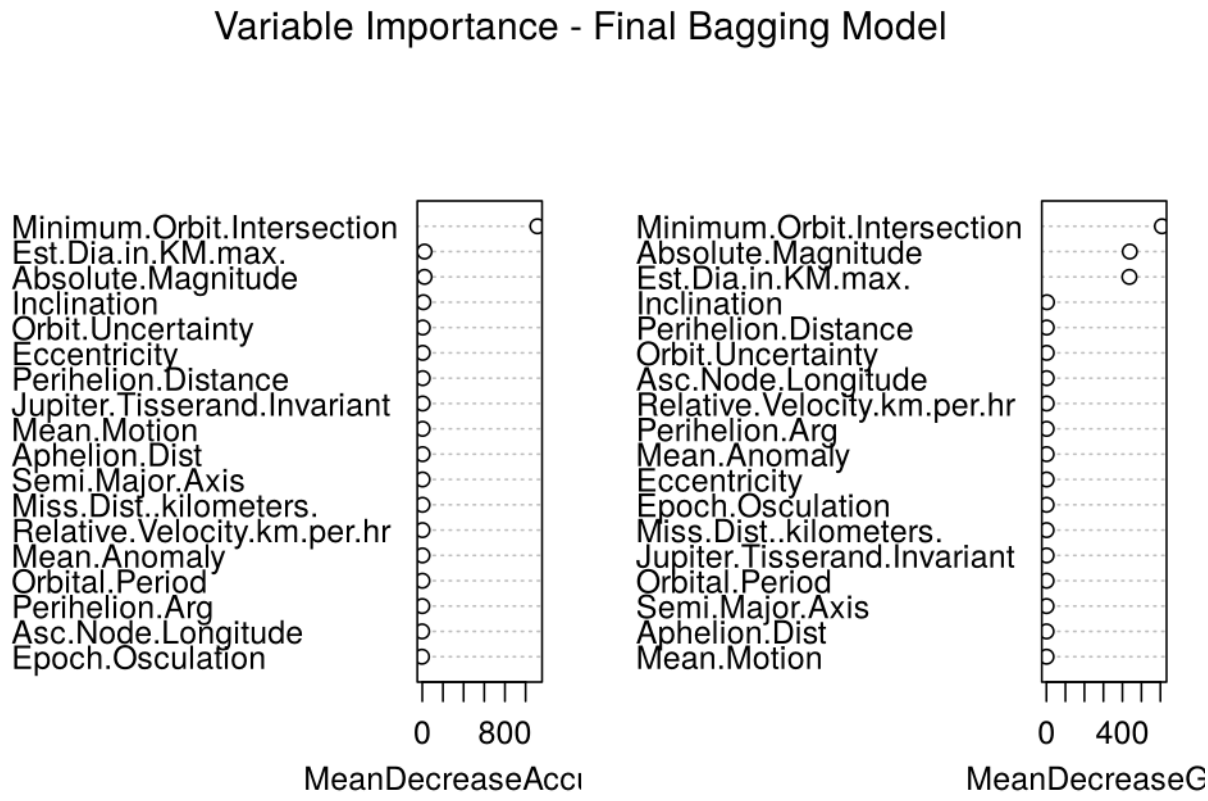
```r
bag_roc <- roc(response = comet_test$Hazardous, predictor = bag_pred_prob,
               levels = c("FALSE", "TRUE"), direction = "<", quiet = TRUE)
bag_auc <- as.numeric(auc(bag_roc))
bag_auc
```

```
## [1] 0.9987048
```

```r
plot(bag_roc, col = "blue", main = "ROC Curve - Bagging Model")
```




```r
varImpPlot(final_bag, main = "Variable Importance - Final Bagging Model")
```



6 Model Comparison

6.1 Confusion Matrices

```
rf_cm <- confusionMatrix(pred_class_3, comet_test$Hazardous, positive = "TRUE")

cat("=== Random Forest - Test Confusion Matrix ===\n"); print(rf_cm$table)
```

```
## === Random Forest - Test Confusion Matrix ===
```

```
##            Reference
## Prediction FALSE TRUE
##      FALSE  6581    8
##      TRUE      5  491
```

```
cat("\n=== Bagging - Test Confusion Matrix ===\n");     print(bag_cm$table)
```

```
##
## === Bagging - Test Confusion Matrix ===
```

```
##            Reference
## Prediction FALSE TRUE
##      FALSE  6350    2
##      TRUE    236  497
```

```
cat("\n=== Boosting - Test Confusion Matrix ===\n");    print(cm_boost_test$table)
```

```
##
```

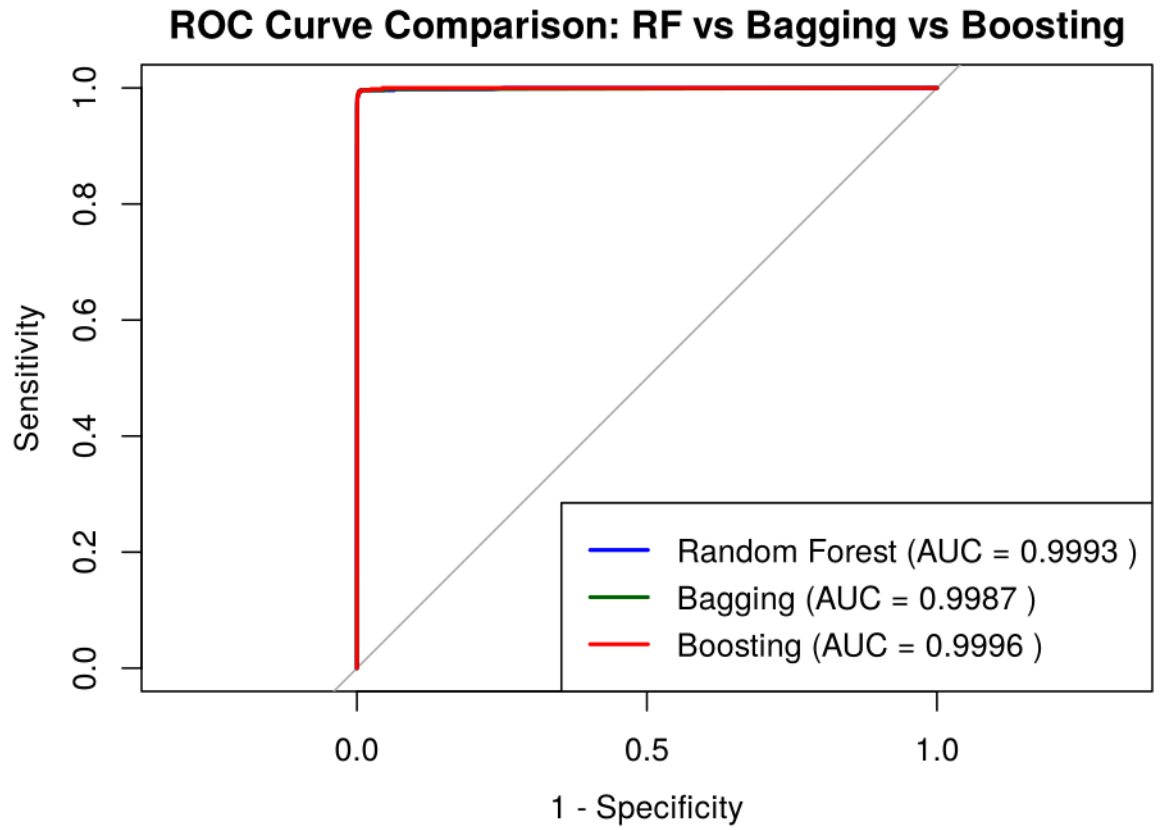
```
## === Boosting - Test Confusion Matrix ===
```

```
##           Reference
## Prediction   No  Yes
##        No  6580    8
##        Yes    6  491
```

6.2 ROC Curve Comparison

```
roc_rf  <- roc(comet_test$Hazardous, pred_probs_3[, "TRUE"], levels = c("FALSE", "TRUE"), quiet = TRUE)
roc_bag <- roc(comet_test$Hazardous, bag_pred_prob,          levels = c("FALSE", "TRUE"), quiet = TRUE)

plot(roc_rf, col = "blue", lwd = 2,
     main = "ROC Curve Comparison: RF vs Bagging vs Boosting", legacy.axes = TRUE)
lines(roc_bag,   col = "darkgreen", lwd = 2)
lines(roc_boost, col = "red",       lwd = 2)
legend("bottomright",
       legend = c(paste("Random Forest (AUC =", round(auc(roc_rf), 4), ")"),
                  paste("Bagging (AUC =",       round(auc(roc_bag), 4), ")"),
                  paste("Boosting (AUC =",      round(auc(roc_boost), 4), ")")),
       col = c("blue", "darkgreen", "red"), lwd = 2)
```



6.3 Metrics Comparison Table

```
ens_row <- function(cm, auc_value) {
  c(Accuracy    = round(cm$overall["Accuracy"], 4),
```

```
    Sensitivity = round(cm$byClass["Sensitivity"], 4),
    Specificity = round(cm$byClass["Specificity"], 4),
    Kappa       = round(cm$overall["Kappa"], 4),
    F1          = round(cm$byClass["F1"], 4),
    AUC         = round(auc_value, 4),
    FN          = cm$table[1, 2],
    FP          = cm$table[2, 1])
}

comparison_table <- data.frame(
  Model = c("Random Forest", "Bagging", "Boosting"),
  rbind(ens_row(rf_cm, as.numeric(auc(roc_rf))),
        ens_row(bag_cm, bag_auc),
        ens_row(cm_boost_test, as.numeric(ROC_AUC)))
)
names(comparison_table) <- c("Model", "Accuracy", "Sensitivity", "Specificity",
                             "Kappa", "F1", "AUC", "FN", "FP")

kable(comparison_table, caption = "Final Model Comparison: Test Set Performance", align = "c") %>%
  kable_styling(position = "center", full_width = FALSE)
```

Table 7: Final Model Comparison: Test Set Performance

Model	Accuracy	Sensitivity	Specificity	Kappa	F1	AUC	FN	FP
Random Forest	0.9982	0.984	0.9992	0.9859	0.9869	0.9993	8	5
Bagging	0.9664	0.996	0.9642	0.7891	0.8068	0.9987	2	236
Boosting	0.9980	0.984	0.9991	0.9849	0.9859	0.9996	8	6

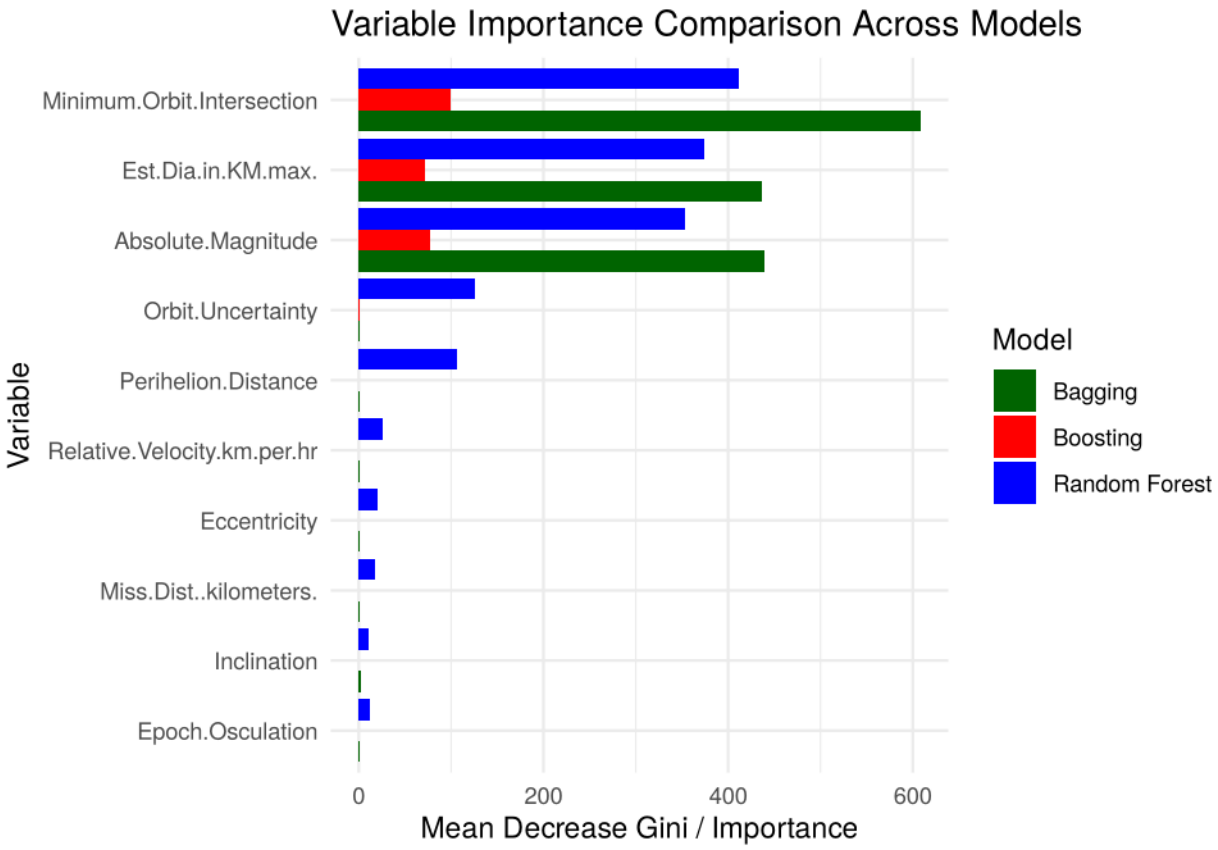
6.4 Variable Importance Comparison

```
rf_imp <- importance(comet_rf_model_tune_2) |> as.data.frame() |>
  rownames_to_column("Variable") |> dplyr::select(Variable, MeanDecreaseGini) |>
  mutate(Model = "Random Forest")
bag_imp <- importance(final_bag) |> as.data.frame() |>
  rownames_to_column("Variable") |> dplyr::select(Variable, MeanDecreaseGini) |>
  mutate(Model = "Bagging")
boost_imp <- varImp(model_boost)$importance |> as.data.frame() |>
  rownames_to_column("Variable") |> rename(MeanDecreaseGini = Overall) |>
  mutate(Model = "Boosting")

all_imp  <- rbind(rf_imp, bag_imp, boost_imp)
top_vars <- all_imp |> group_by(Variable) |>
  summarise(AvgImportance = mean(MeanDecreaseGini)) |>
  arrange(desc(AvgImportance)) |> slice_head(n = 10) |> pull(Variable)

all_imp |>
  filter(Variable %in% top_vars) |>
  mutate(Variable = reorder(Variable, MeanDecreaseGini)) |>
  ggplot(aes(x = MeanDecreaseGini, y = Variable, fill = Model)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Variable Importance Comparison Across Models",
       x = "Mean Decrease Gini / Importance", y = "Variable") +
```

```
  theme_minimal() +
  scale_fill_manual(values = c("Random Forest" = "blue", "Bagging" = "darkgreen", "Boosting" = "red"))
```



6.5 All Models on the Test Set

Step 2 sensitivity and specificity are taken at each model's ROC-optimal point.

```
predictors_test_hco_data <- as.matrix(test_hco_data[, names(test_hco_data) != "Hazardous"])
target_test_hco          <- as.numeric(as.character(test_hco_data$Hazardous))

pred_baseline_test <- predict(baseline_model_hco_data, newdata = test_hco_data, type = "response")
pred_stepwise_test <- predict(stepwise_hco_data,       newdata = test_hco_data, type = "response")
pred_best_test     <- predict(fit_best,                newdata = test_hco_data, type = "response")
pred_ridge_test    <- predict(ridge_5_fold_hco_data, newx = predictors_test_hco_data,
                              s = ridge_lambda_min_hco_data, type = "response")[, 1]
pred_lasso_test    <- predict(lasso_5_fold_hco_data, newx = predictors_test_hco_data,
                              s = lasso_lambda_min_hco_data, type = "response")[, 1]

roc_baseline_test <- roc(target_test_hco, pred_baseline_test, quiet = TRUE)
roc_stepwise_test <- roc(target_test_hco, pred_stepwise_test, quiet = TRUE)
roc_best_test     <- roc(target_test_hco, pred_best_test,     quiet = TRUE)
roc_ridge_test    <- roc(target_test_hco, pred_ridge_test,    quiet = TRUE)
roc_lasso_test    <- roc(target_test_hco, pred_lasso_test,    quiet = TRUE)

get_best_coords <- function(roc_obj, metric) {
  round(as.numeric(coords(roc_obj, "best", ret = metric, transpose = FALSE)[1, ]), 4)
```

```
}

step2_rocs <- list(roc_baseline_test, roc_stepwise_test, roc_best_test, roc_ridge_test, roc_lasso_test)

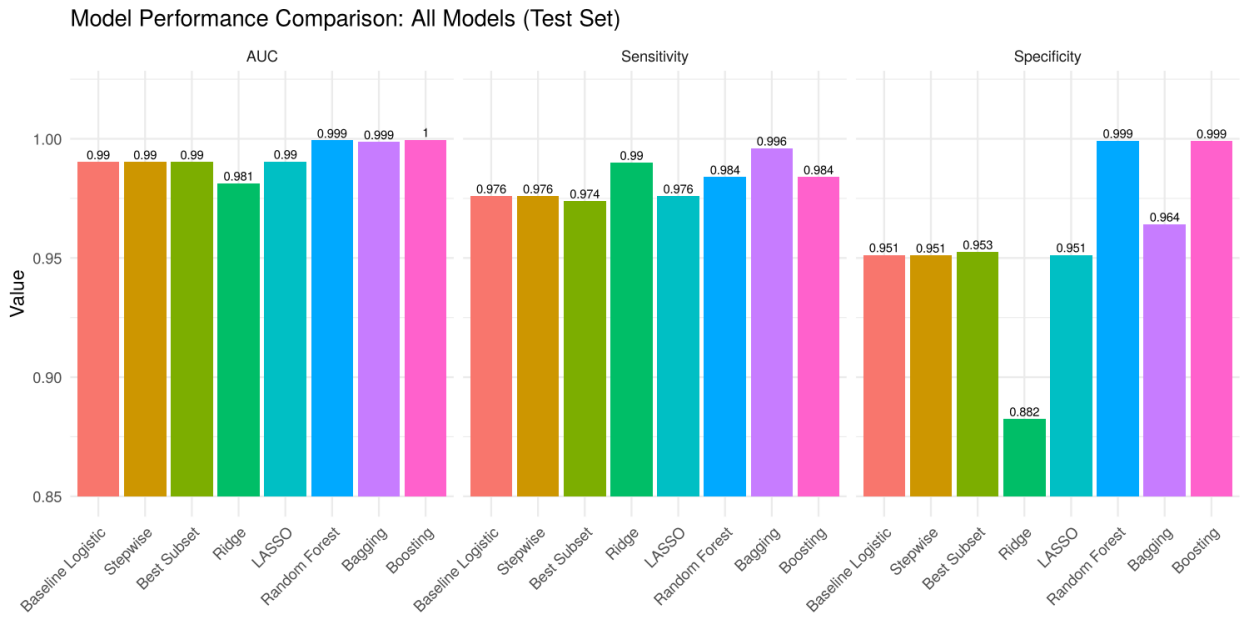
all_models_table <- data.frame(
  Model = c("Baseline Logistic", "Stepwise", "Best Subset", "Ridge", "LASSO",
            "Random Forest", "Bagging", "Boosting"),
  AUC = c(sapply(step2_rocs, function(r) round(as.numeric(auc(r)), 4)),
          round(as.numeric(auc(roc_rf)), 4), round(bag_auc, 4), as.numeric(ROC_AUC)),
  Sensitivity = c(sapply(step2_rocs, get_best_coords, metric = "sensitivity"),
                  round(rf_cm$byClass["Sensitivity"], 4),
                  round(bag_cm$byClass["Sensitivity"], 4),
                  round(cm_boost_test$byClass["Sensitivity"], 4)),
  Specificity = c(sapply(step2_rocs, get_best_coords, metric = "specificity"),
                  round(rf_cm$byClass["Specificity"], 4),
                  round(bag_cm$byClass["Specificity"], 4),
                  round(cm_boost_test$byClass["Specificity"], 4))
)
kable(all_models_table, caption = "All Models: Test Set", align = "c") %>%
  kable_styling(position = "center", full_width = FALSE)
```

Table 8: All Models: Test Set

Model	AUC	Sensitivity	Specificity
Baseline Logistic	0.9903	0.9760	0.9513
Stepwise	0.9903	0.9760	0.9511
Best Subset	0.9903	0.9739	0.9525
Ridge	0.9813	0.9900	0.8825
LASSO	0.9904	0.9760	0.9513
Random Forest	0.9993	0.9840	0.9992
Bagging	0.9987	0.9960	0.9642
Boosting	0.9996	0.9840	0.9991

```
all_models_long <- pivot_longer(all_models_table, cols = -Model, names_to = "Metric", values_to = "Valu
all_models_long$Model <- factor(all_models_long$Model, levels = all_models_table$Model)

ggplot(all_models_long, aes(x = Model, y = Value, fill = Model)) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = round(Value, 3)), vjust = -0.3, size = 2.5) +
  facet_wrap(~ Metric, ncol = 3) +
  scale_y_continuous(limits = c(0.85, 1.02), oob = scales::squish) +
  labs(title = "Model Performance Comparison: All Models (Test Set)", x = NULL, y = "Value") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1), legend.position = "none")
```



7 Session Info

```
sessionInfo()
```

```
## R version 4.3.3 (2024-02-29)
## Platform: x86_64-pc-linux-gnu (64-bit)
## Running under: Ubuntu 24.04.5 LTS
##
## Matrix products: default
## BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
## LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
##
## locale:
## [1] C
##
## time zone: America/New_York
## tzcode source: system (glibc)
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] GGally_2.2.1         corrplot_0.92        tibble_3.2.1
##  [4] scales_1.3.0         gbm_2.1.8.1          MASS_7.3-60.0.1
##  [7] leaps_3.1            glmnet_4.1-8         Matrix_1.6-5
## [10] pROC_1.18.5          randomForest_4.7-1.1 caret_6.0-94
## [13] lattice_0.22-5       readxl_1.4.3         kableExtra_1.4.0
## [16] knitr_1.45           tidyr_1.3.1          ggplot2_3.4.4
## [19] dplyr_1.1.4
##
## loaded via a namespace (and not attached):
##  [1] tidyselect_1.2.0     viridisLite_0.4.2    timeDate_4032.109
```

```
##  [4] farver_2.1.1         fastmap_1.1.1        digest_0.6.34
##  [7] rpart_4.1.23         timechange_0.3.0     lifecycle_1.0.4
## [10] survival_3.5-8       magrittr_2.0.3       compiler_4.3.3
## [13] rlang_1.1.3          sass_0.4.8           tools_4.3.3
## [16] utf8_1.2.4           yaml_2.3.8           data.table_1.14.10
## [19] labeling_0.4.3       RColorBrewer_1.1-3   plyr_1.8.9
## [22] xml2_1.3.6           withr_2.5.0          purrr_1.0.2
## [25] nnet_7.3-19          grid_4.3.3           stats4_4.3.3
## [28] fansi_1.0.5          e1071_1.7-14         colorspace_2.1-0
## [31] future_1.33.1        globals_0.16.2       iterators_1.0.14
## [34] cli_3.6.2            rmarkdown_2.25       generics_0.1.3
## [37] rstudioapi_0.15.0    future.apply_1.11.1  reshape2_1.4.4
## [40] proxy_0.4-27         cachem_1.0.8         stringr_1.5.1
## [43] splines_4.3.3        parallel_4.3.3       cellranger_1.1.0
## [46] vctrs_0.6.5          hardhat_1.3.1        jsonlite_1.8.8
## [49] listenv_0.9.1        systemfonts_1.0.5    foreach_1.5.2
## [52] gower_1.0.1          jquerylib_0.1.4      recipes_1.0.9
## [55] glue_1.7.0           parallelly_1.37.1    ggstats_0.5.1
## [58] codetools_0.2-19     shape_1.4.6          lubridate_1.9.3
## [61] stringi_1.8.3        gtable_0.3.4         munsell_0.5.0
## [64] pillar_1.9.0         htmltools_0.5.7      ipred_0.9-14
## [67] lava_1.7.3           R6_2.5.1             evaluate_0.23
## [70] highr_0.10           bslib_0.6.1          class_7.3-22
## [73] Rcpp_1.0.12          svglite_2.1.3        nlme_3.1-164
## [76] prodlim_2023.08.28   mgcv_1.9-1           xfun_0.41
## [79] pkgconfig_2.0.3      ModelMetrics_1.2.2.2
```